

软件工程基础

第6章 软件测试

杨溢龙

北航软件学院

目　录

Contents

软件工程基础

软件测试

Part 1 软件缺陷

Part 2 软件测试概论

Part 3 软件测试分类

Part 4 软件测试过程

Part 5 软件测试管理



Part 1

软件缺陷

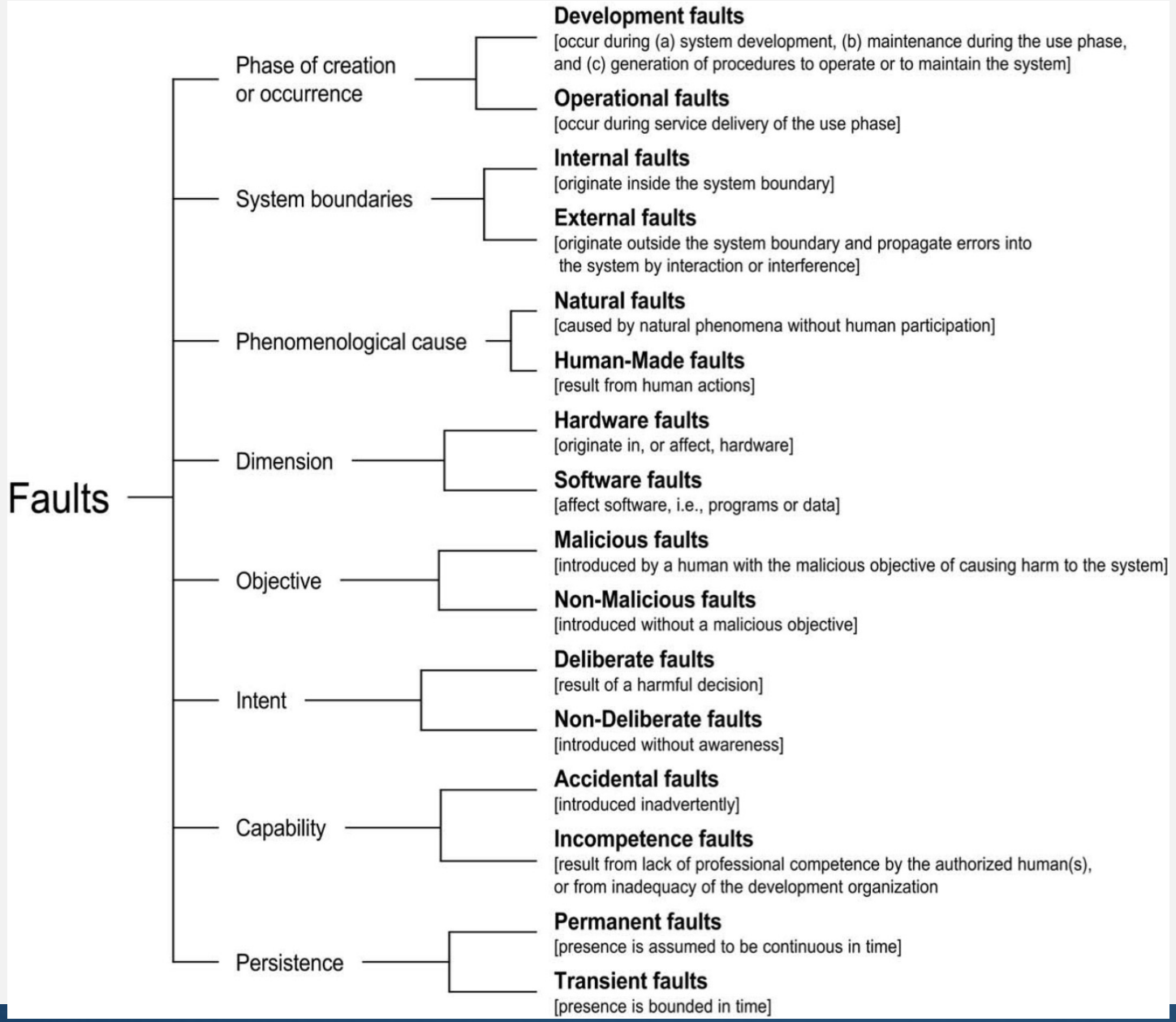
Part 1 编码缺陷

- 缺陷的基本概念
- 缺陷的描述
- 缺陷的应对方法

Part 1 编码缺陷与调试

- 缺陷的基本概念
- 缺陷的描述
- 缺陷的应对方法

故障（Fault)



IEEE TRANSACTIONS ON DEPENDABLE AND SECURE COMPUTING, VOL. 1, NO. 1, JANUARY-MARCH 2004

Basic Concepts and Taxonomy of Dependable and Secure Computing


Algirdas Avižienis


Jean-Claude Laprie

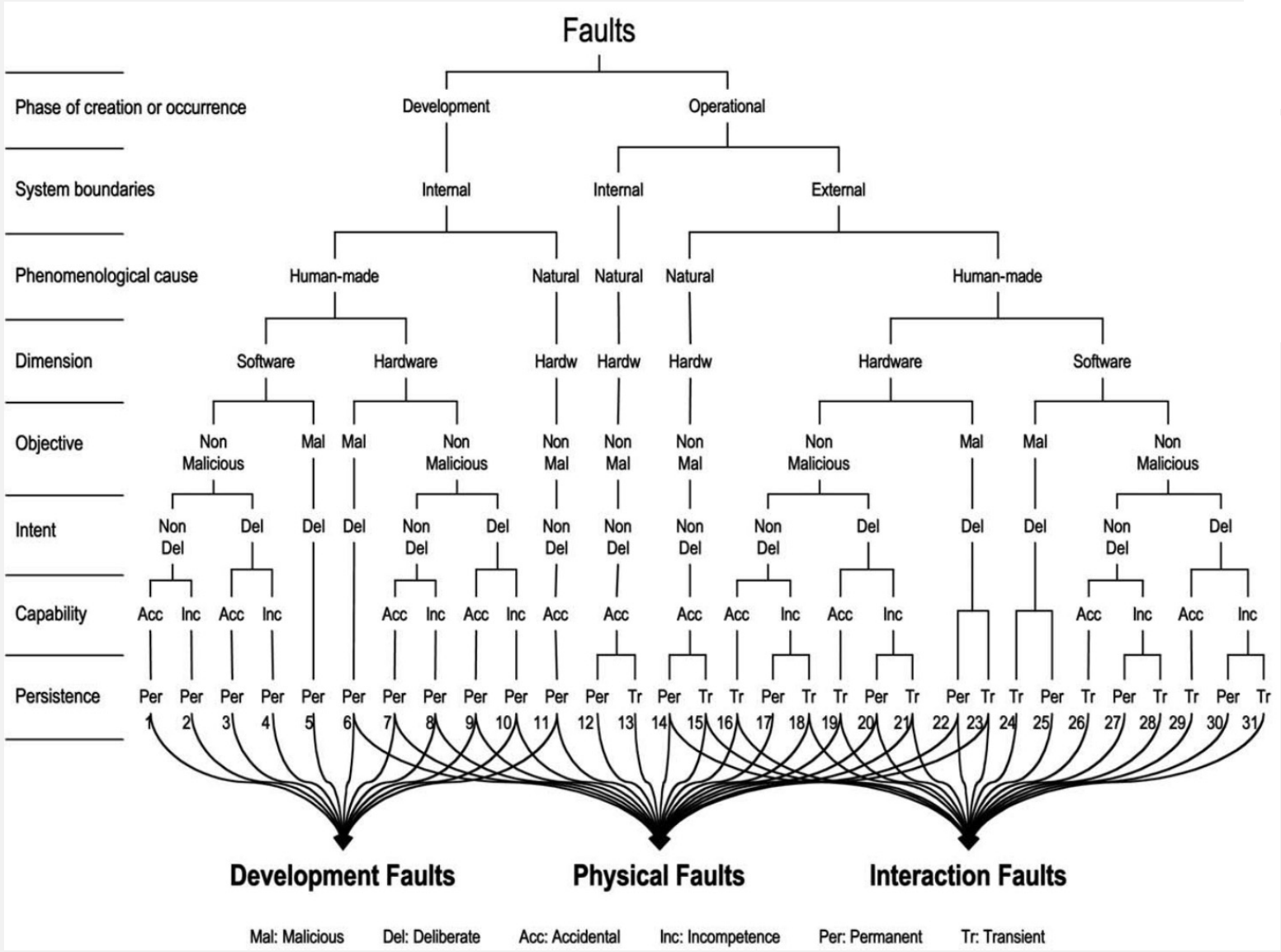

Brian Randell


Carl Landwehr

Fault

- a **broken part** or **weakness** in a machine or system
  - 系统受损或存在的缺陷
- The **adjudged (esp. officially announced) (官宣的)** or **hypothesized (假设的) cause** of an error.

故障（Fault）分类



IEEE TRANSACTIONS ON DEPENDABLE AND SECURE COMPUTING, VOL. 1, NO. 1, JANUARY-MARCH 2004

Basic Concepts and Taxonomy of Dependable and Secure Computing



分类标准

- 发生阶段（开发，运维）
- 系统边界（内部，外部）
- 现象（自然，人为）
- 分类（软件，硬件）
- 目标（恶意的，无意的）
- 目的（故意，非故意的）
- 持久性（永久的，瞬时的）

Defects/Bugs是其中一种

软件缺陷 (Software Defects / Bugs)

- **软件缺陷是指软件制品中存在不正确的软件描述和实现**
  - ✓存在缺陷的软件制品不仅包括**程序代码**，而且还包括需求和设计的**模型和文档**
  - ✓软件缺陷**产生于软件开发全过程**，只有有人介入的地方就有可能产生软件缺陷
  - ✓**任何人**都有可能在软件开发过程中犯错误，进而引入软件缺陷
  - ✓无论是高层的需求分析和软件架构缺陷还是底层的详细设计缺陷，它们**最终都会反映在程序代码**之中，导致程序代码存在缺陷

故障 Fault → 错误 Error

Error: **external state** of the system **deviates from** the **correct** service state. Such **deviation (偏差)** is called an error.

- **程序错误：存在故障的程序代码在运行过程中会产生不正确或不符合期望的运行状态**
  - ✓ 经过计算后某个变量的取值不正确
  - ✓ 接收到的消息内容不正确
  - ✓ 打开一个非法的文件
- **一些情况之下引发程序报错**





A **fault** is **active** when it causes an error, otherwise it is **dormant (休眠的)**

错误 Error → 失效 Failure

A failure is an **event** that occurs when the **delivered service** deviates from **correct service**.
A service fails either because it does **not comply with** the functional specification, or because this specification did not **adequately describe** the system function.

- **运行错误的程序无法为用户提供所需的功能和行为，在此情况下我们称程序出现了失效（Failure）**
  - ✓ 如用户无法正常登录到系统中
  - ✓ 无法正确地分析出老人是否处于摔倒的状态等等
- **程序错误的根源在于程序中存在故障，程序的错误运行必然导致软件失效**

示例：软件中存在缺陷，导致软件失效

高校课程实践社区

当前位置：主页 > 课程实践平台 > 软件工程

Trustie Course

ID:342

配置 关闭

软件工程

教师（2）| 学生（21）| 资源（2）

主讲教师： 毛新军
学时总数： 54 学时
课程学期： 2015 秋季学期
开设单位： 国防科学技术大学

动态 (15)

课程作业 (0) +发布作业

课程通知 (0) +发布通知

资源库 (2) +上传文件

讨论区 (7) +发布新帖

留言 (0)

问卷调查 (1) +新建问卷

课内搜索 全站搜索

上传： 课件 | 软件 | 媒体 | 代码 | 其他

共有 2 个资源

按 时间 / 下载次数 / 引用次数 排序

课件×2

2 软件与软件工程.pdf 选入我的其他课程 公开 预览

文件大小：4.475 MB

1天之前 | 下载2 | 引用0 删除

课件 x + 添加标签

1 课程介绍与要求.pdf 选入我的其他课程 公开 预览

文件大小：6.755 MB

6 天之前 | 下载10 | 引用0 删除

课件 x + 添加标签

✓ 上传资源后资源数量没有变化
✓ 查找以前上传的资源找不到

示例：软件中存在缺陷，导致软件失效

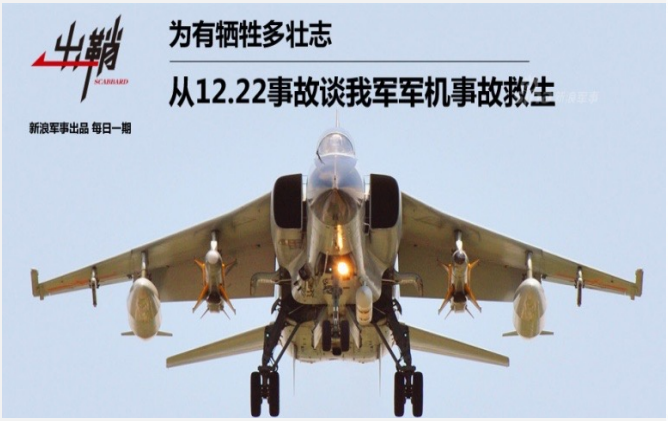


软件缺陷的危害

- 无法满足要求
- 不能正常工作
- 引发安全事故
- 影响人员安全
- 产生经济损失
- ……



波音737 MAX事件





原软件用于教练飞机某参数屏显范围±100，重用于新战斗机，该参数屏显范围应该为±300!

尽可能减少软件缺陷非常重要

软件缺陷不可避免

- **人总是会犯错误的**
  - ✓软件工程师、用户等
  - ✓软件系统太复杂
- **程序缺陷来自多个源头**
  - ✓需求、设计、编码活动
  - ✓模型、文档、程序制品
- **缺陷成常态化**
  - ✓对于复杂软件系统而言缺陷不可避免
  - ✓很难做到无缺陷的软件

你所使用的软件中是否发现有缺陷?



Open Question

Points:10

设置

❑如何应对软件缺陷?

- ✓如何避免和减少缺陷?
- ✓如何发现和修复缺陷?



Answer

Part 1 编码缺陷

- 缺陷的基本概念
- 缺陷的描述
- 缺陷的应对方法

软件缺陷的描述（1/2）

- **标识符**
  - ✓每个软件缺陷都被给予一个唯一的标识符。
- **类型**
  - ✓说明软件缺陷的类型，如**需求缺陷、设计缺陷、代码缺陷**
  - ✓代码缺陷还可以进一步区分为是**逻辑缺陷、计算缺陷、判断缺陷**
- **严重程度**
  - ✓**危急程度**是指缺陷会**影响**软件的**正常运行甚至危及用户安全**
  - ✓**严重程度**的缺陷是指会导致软件**丧失某些重要功能**，或出现错误
  - ✓**一般程度**的缺陷是指缺陷会使得软件**丧失**某些**次要的功能**
  - ✓**轻微程度**是指缺陷会导致软件出现**小毛病**，但**不影响正常运行**

软件缺陷的描述（2/2）

- **症状**：软件缺陷所引发的程序错误是什么，有何运行表现
- **修复优先级**：缺陷应该被修复的优先程度，包括：非常紧迫、紧迫、一般和不紧迫等几种
- **状态**：描述缺陷处理的进展状态，如已经安排人员来处理、正在修复、修复已经完成等。
- **发现者**：谁发现了软件缺陷。
- **发现时机**：什么状况下发现的软件缺陷
- **源头**：软件缺陷源头在哪里，如软件文档的哪一个部分
- **原因**：说明导致软件缺陷的原因是什么

Part 1 编码缺陷

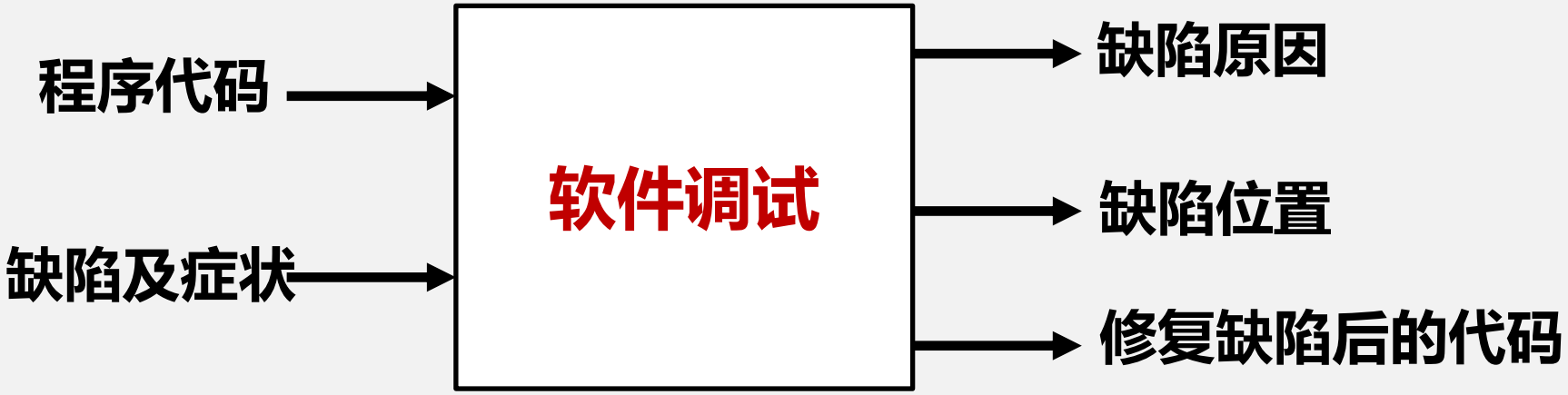
- 缺陷的基本概念
- 缺陷的描述
- 缺陷的应对方法

软件缺陷的应对方法（1/2）

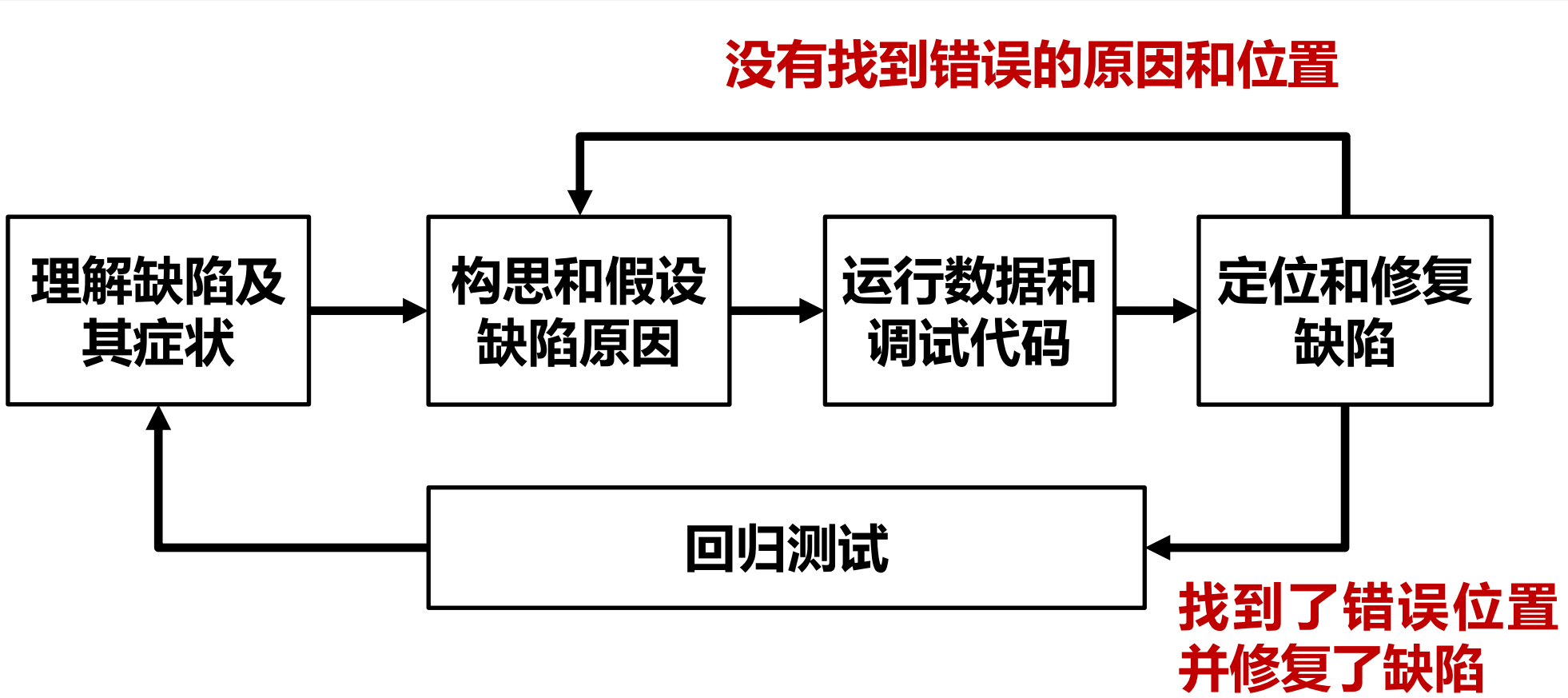
- **发现缺陷**
  - ✓ 通过有效的**技术和管理手段来发现这些软件缺陷**
  - ✓ 例如，**制定和实施软件质量保证计划**、开展软件文档和模型的**评审**、程序代码的**走查**、软件测试等工作。它们都可以帮助软件工程师找到潜藏在文档、模型和代码中的软件缺陷
- **修复缺陷**
  - ✓ 通过一系列的手段来修复缺陷
  - ✓ 采用程序调试等手段来**找到缺陷的原因、定位缺陷的位置**，进而**修改存在缺陷的程序代码**，将软件缺陷从软件制品中**移除出去**

程序调试

- **软件调试就是要基于程序代码，确定软件缺陷的原因、定位缺陷的位置，从而知道那里错了、如何修复缺陷**
  - ✓程序员需要花费大量的时间和精力用于软件调试
  - ✓软件调试通过运行目标软件系统的程序代码，找到缺陷的代码位置、明确软件错误的具体原因，从而开展缺陷修复工作



调试的步骤


没有找到错误的原因和位置
理解缺陷及其症状
构思和假设缺陷原因
运行数据和调试代码
定位和修复缺陷
回归测试
找到了错误位置并修复了缺陷

软件缺陷的应对方法（2/2）

❑ 容忍缺陷（Tolerance）

- ✓ 增强软件的缺陷容忍度，借助于**软件容错机制和技术**，允许软件出现错误，但是在**出现错误时软件仍然能够正常的运行**
- ✓ 在高可靠软件系统的开发过程中，软件工程师通常需要提供**容错模块和代码**。显然这会增加软件开发的**复杂度和冗余度**

❑ 预防缺陷（Prevention）

- ✓ 通过运用各种软件工程技术、方法和管理手段，在软件开发过程中**预防和避免软件缺陷，减少软件缺陷的数量，降低软件缺陷的严重程度**
- ✓ 采用**结对编程**、**严格的过程管理**、必要的技术培训、CASE工具的使用等手段，起到**预防缺陷的目的**

缺陷管理工具

- 工具： PingCode 、ClickUp、 Jira、Redmine Tool、Redmine Tool、Zoho Bug Tracker、Asana、nTask、Bugzilla、Mantis BT、BugLog HQ

北京航空航天大學
BEIHANG UNIVERSITY



Part 2

软件测试概论

Part 2 软件测试概论

- 软件测试的概念
- 软件测试的目的
- 软件测试的本质
- 软件测试的过程
- 软件测试的原则

Part 2 软件测试概论

- 软件测试的概念
- 软件测试的目的
- 软件测试的本质
- 软件测试的过程
- 软件测试的原则

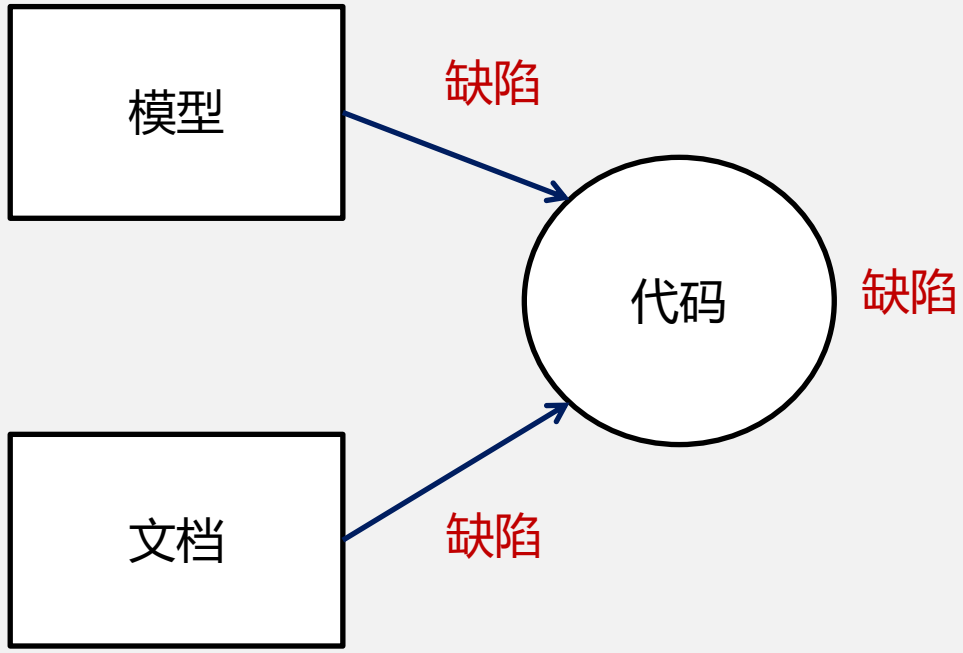
软件测试概论

- ✓ 软件产品与其他产品最大的不同是其复杂性，从而导致发现并修正软件缺陷的成本特别大。
- ✓ 发现软件缺陷的过程叫做软件测试
- ✓ 修正软件缺陷的过程叫做软件纠错或软件维护

- ✓ 什么是测试?
  - □ 测试的英文单词为Testing，即检验或考试之意。
  - □ 【定义】所谓测试，就是通过一定的方法或工具，对被测试对象进行检验，目的是发现被测试对象具有某种属性或者存在某些问题。

在程序代码中找出软件缺陷

- **程序是运行软件的载体**
  - ✓通过执行代码运行软件
  - ✓通过软件运行发现缺陷
- **程序是软件缺陷的载体**
  - ✓缺陷分布在模型、文档和代码中
  - ✓最终会**反映在程序代码**上



Open Question

Points:10

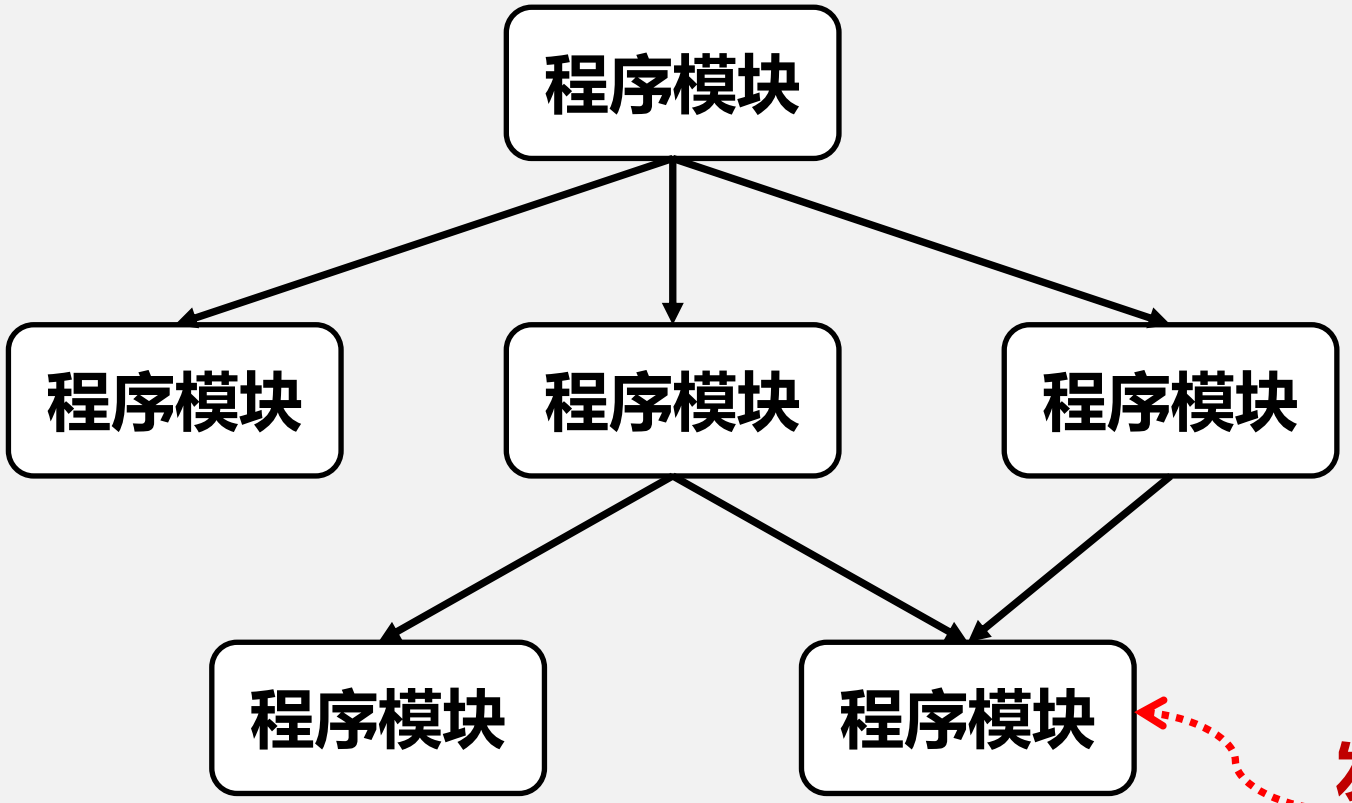
设置

❑软件缺陷会“潜伏”在程序代码的哪些地方?



Answer

程序构成及其缺陷的潜在位置



- 程序模块内部
- 程序模块接口
- 程序模块间的交互
- 整个软件系统

发现和消除基本模块单元中缺陷

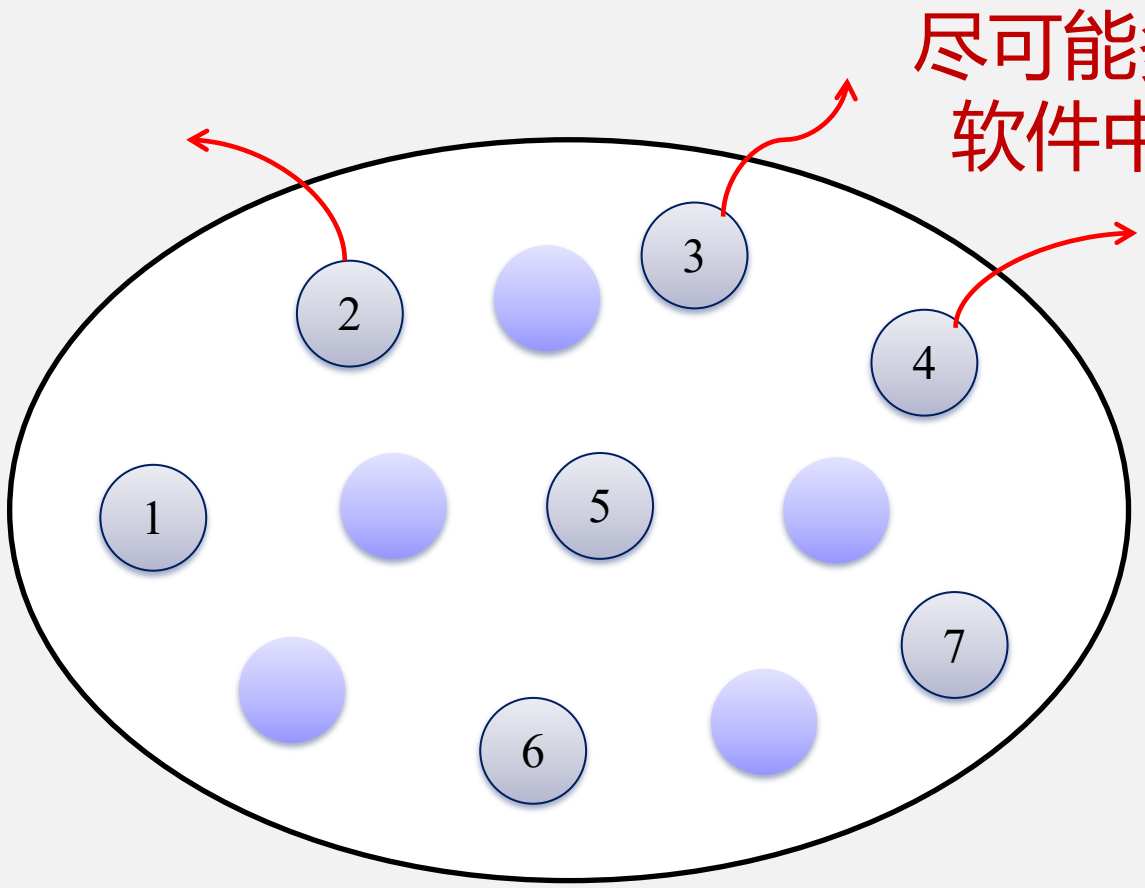
Part 1 软件测试概论

- 软件测试的概念
- 软件测试的目的
- 软件测试的本质
- 软件测试的过程
- 软件测试的原则

软件测试的目的

- **目的**
  - ✓**发现软件中的缺陷**
  - ✓最大限度、尽可能多的找到缺陷
- **功效**
  - ✓发现的缺陷越多 ➔ 软件中遗留的缺陷越少
    ➔ 交付的软件质量越高 ➔ 后期维护工作量就越少

软件测试



软件缺陷可能隐藏的比较深，难以发现

❑运行软件或模拟软件的执行，发现软件缺陷

Open Question

Points:10

设置

采用这种方法能把软件中的所有缺陷都找出来吗?



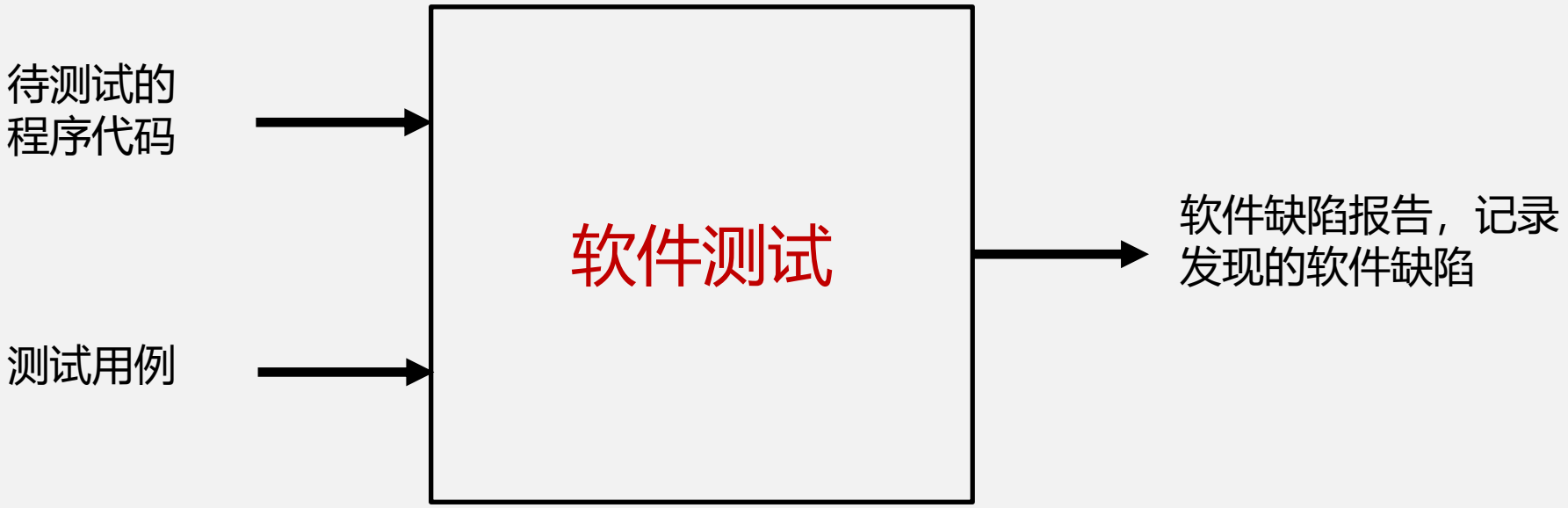


Answer

Part 1 软件测试概论

- 软件测试的概念
- 软件测试的目的
- 软件测试的本质
- 软件测试的过程
- 软件测试的原则

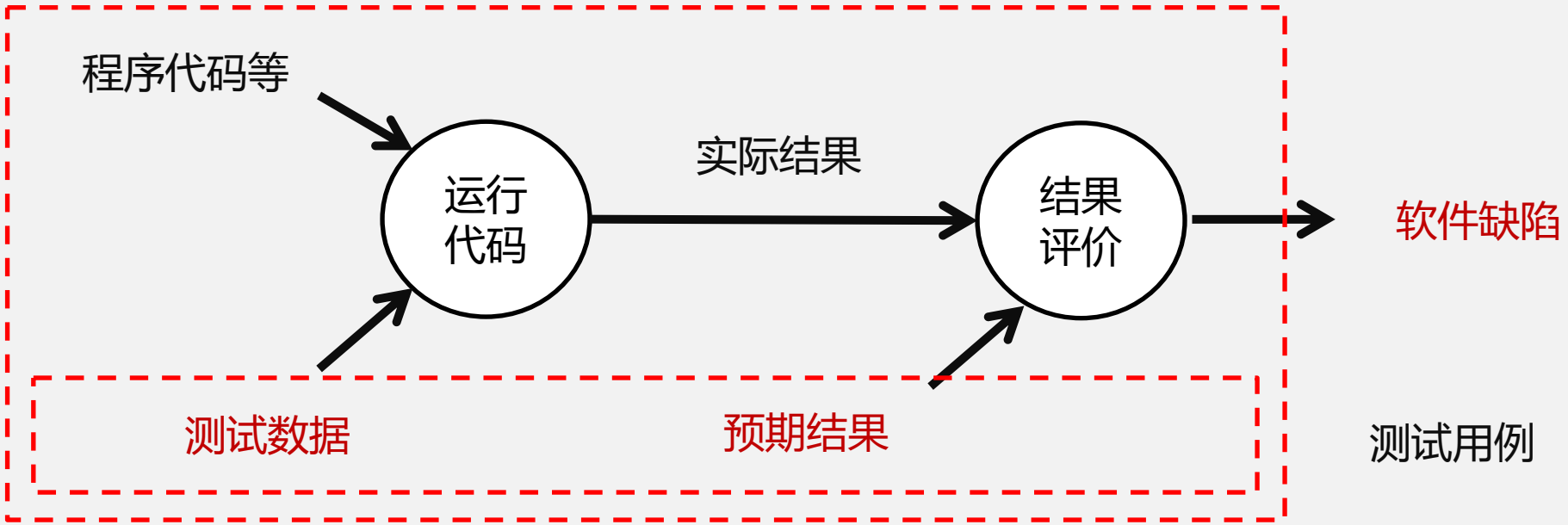
软件测试的输入与输出



软件测试的本质

❑程序本质上是对数据的处理

✓设计数据(测试用例) ➔ 运行测试用例(程序来处理数据) ➔ 判断运行结果(是否符合预期结果)



为软件测试而设计的数据称为测试用例(Test Case)

测试用例

- **测试用例是一个四元组**
  - ✓**输入数据**：交由待测试程序代码进行处理的数据
  - ✓**前置条件**：程序处理输入数据的运行上下文，即要满足前置条件
  - ✓**测试步骤**：程序代码对输入数据的处理可能涉及到一系列的步骤，其中的某些步骤需要用户的进一步输入
  - ✓**预期输出**：程序代码的预期输出结果

示例：测试用例的设计

- **“用户登录”模块单元的测试用例设计**
  - ✓**输入数据**：用户账号=“admin”，用户密码=“1234”
  - ✓**前置条件（上下文）**：用户账号“admin”是一个尚未注册的非法账号，也即“T_User”表中没有名为“admin”的用户账号。
  - ✓**测试步骤**：首先清除“T_User”表中名为“admin”的用户账号；其次用户输入“admin”账号和“1234”密码；第三，用户点击界面的确认按钮；最后，系统提示“用户无法登录系统”的信息
  - ✓**预期输出**：系统将提示“用户无法登录系统”的提示信息

Part 1 软件测试概论

- 软件测试的概念
- 软件测试的目的
- 软件测试的本质
- 软件测试的过程
- 软件测试的原则

软件测试的步骤

① 明确待测试对象

✓什么粒度的程序代码

② 设计测试用例

✓ {<Data, Result>}

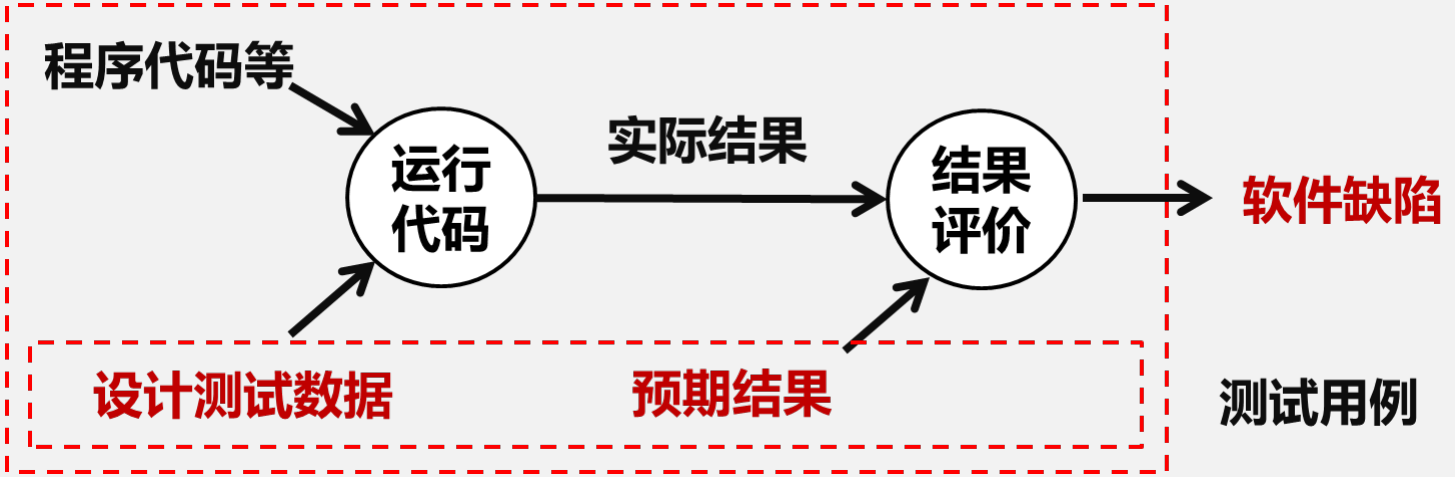
✓可能有许多

③ 运行代码和测试用例

✓输入和处理测试用例

④ 分析运行结果

✓对比运行结果和预期结果，发现问题和缺陷



示例: 软件测试的步骤

- **明确测试对象**
  - ✓完成用户注册功能的程序模块（类）
- **设计测试用例**
  - ✓输入：UserID = xjmao, Psw=se
  - ✓预期：在数据库中有该用户密码的数据条目
- **运行测试用例**
  - ✓运行程序，输入数据
- **分析运行结果**
  - ✓用户数据库表中是否有该新用户的密码信息

Part 1 软件测试概论

- 软件测试的概念
- 软件测试的目的
- 软件测试的本质
- 软件测试的过程
- 软件测试的原则

软件测试原则(1/2)

- **测试应该有计划**
  - ✓在开发初期就应制定**测试计划**，测试计划应该在测试之前就应该制定
- **所有的测试都应该追溯到用户需求**
- **将Pareto原则应用于软件测试**
  - ✓80%的错误都可以在大概20%的模块中找到根源
- **测试应该从“微观”开始，逐步转向“宏观”**
- **穷举测试是不可能的**
- **每个测试用例都必须定义预期的输出或结果**
- **尽量避免程序的开发人员来测试他自己编写的代码**
- **尽量避免程序开发组织测试其自己开发的程序**

软件测试原则(2/2)

- 应详细检查每次测试的结果。
- 测试用例中不仅要说明合法有效的输入条件，还应该描述那些不期望的、非法的输入条件。
- 检查一个程序没有完成希望它做的事情只是测试的一半任务，还应检查程序是否执行了不希望它做的事情。
- 避免随意舍弃任何测试用例，即使非常简单的测试用例。
- 不应在事先假设不会发现错误的情况下来制定测试计划。
- 一个程序模块存在更多错误的可能性与该模块已经发现的错误数量成正比。
- 测试是一个具有相当创新性和智力挑战的活动。

Open Question

Points:10

设置

- 软件测试没有发现缺陷是否意味着软件就没有缺陷？
- 软件测试能够用于证明软件无缺陷吗?



Answer



Part 2

软件测试分类

Part 2 软件测试分类

- 软件测试分类概述
- 软件白盒测试
- 软件黑盒测试

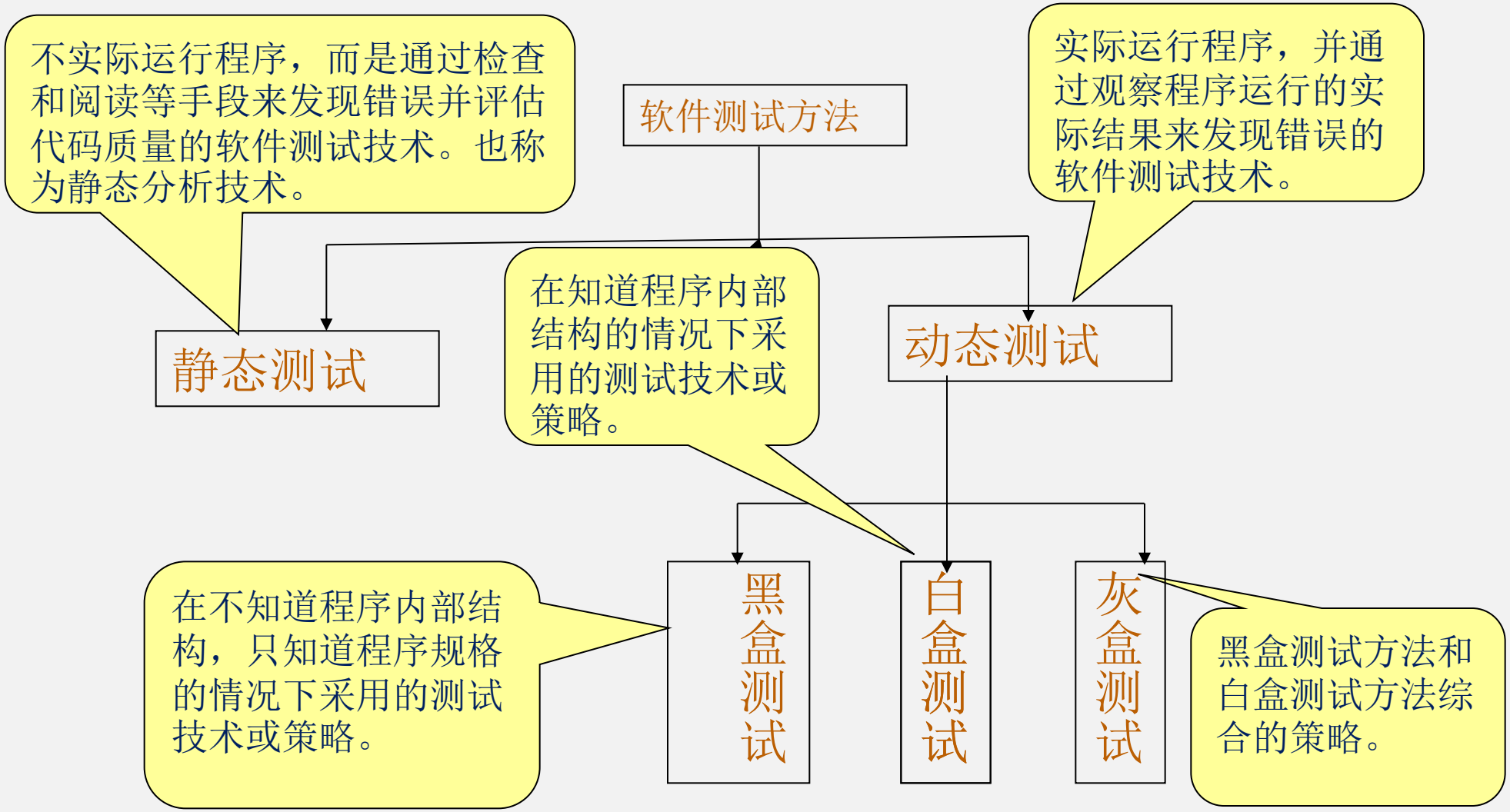
Part 2 软件测试分类

- 软件测试分类概述
- 软件白盒测试
- 软件黑盒测试

软件测试的分类


软件测试
按开发阶段
按测试实施组织
按测试技术
按测试内容
单元测试
集成测试
确认测试
系统测试
验收测试
开发方测试
用户测试
第三方测试
白盒测试
黑盒测试
灰盒测试
符合性测试
验收测试
易用性测试
兼容性测试
可靠性测试
安全性测试
性能测试

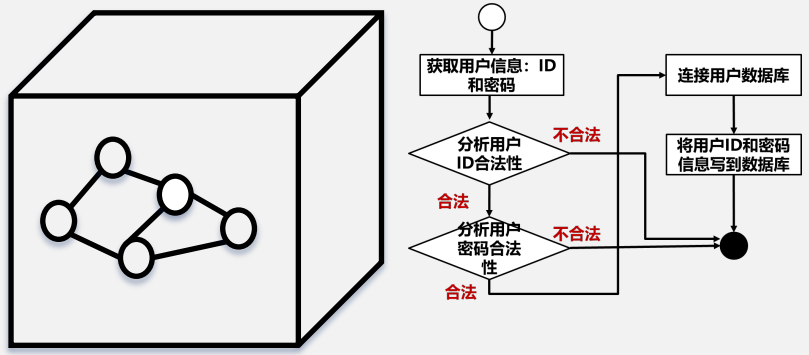
测试技术分类


软件测试方法
静态测试
不实际运行程序，而是通过检查和阅读等手段来发现错误并评估代码质量的软件测试技术。也称为静态分析技术。
动态测试
实际运行程序，并通过观察程序运行的实际结果来发现错误的软件测试技术。
在知道程序内部结构的情况下采用的测试技术或策略。
黑盒测试
在不知道程序内部结构，只知道程序规格的情况下采用的测试技术或策略。
白盒测试
灰盒测试
黑盒测试方法和白盒测试方法综合的策略。

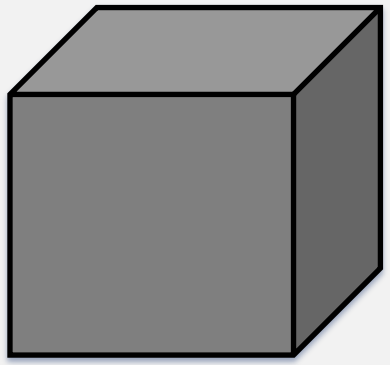
软件测试技术

- 如何设计和运行测试用例

- 白盒测试技术
  - 基于程序**内部的执行流程**来设计测试用例



- 黑盒测试技术
  - 基于程序的**外在功能和接口**来设计测试用例



程序功能

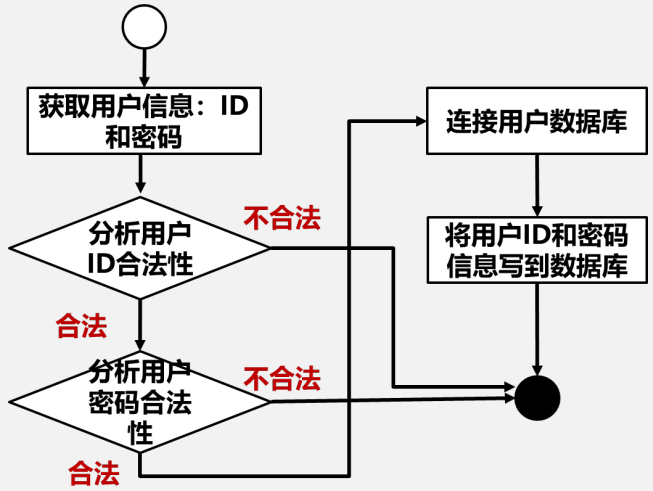
盒子对应于基本模块单元（如类方法、函数、过程等），测试基本对象

Part 2 软件测试分类

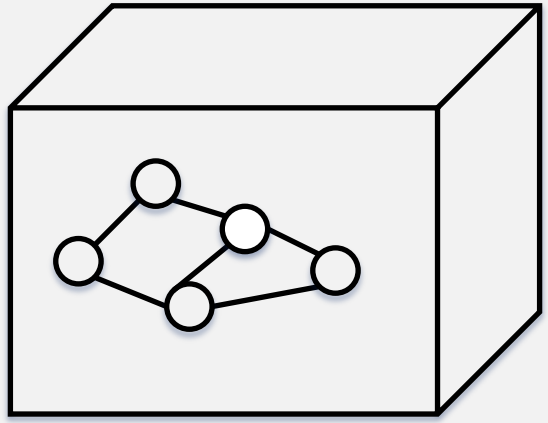
- 软件测试分类概述
- 软件白盒测试
- 软件黑盒测试

软件测试技术-白盒测试

- 设计测试用例思想
  - 根据**程序单元内部工作流程**来设计测试用例
- 发现程序单元缺陷
  - 运行待测试的程序，**检验程序是否按内部工作流程来运行**的，如果不是则存在缺陷
- 特点
  - 必须了解程序的内部工作流程才能设计测试用例

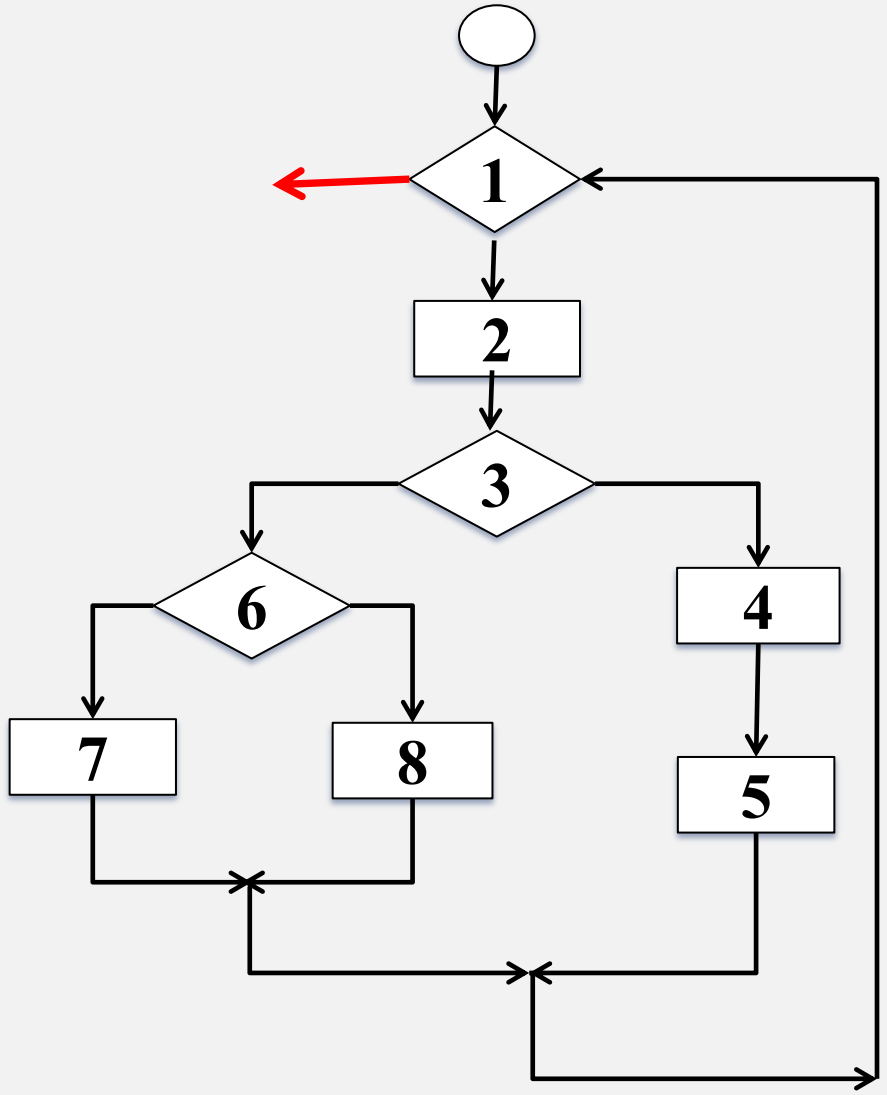


模块内部流程



白盒测试用例设计的指导原则

- 如何设计测试用例?
  - 内部执行流程
  - 生成测试数据
- 设计多少测试用例?
  - 遵循覆盖原则
- 测试用例覆盖准则
  - 语句覆盖
  - 分支覆盖
  - 路径覆盖
  - 基本路径覆盖

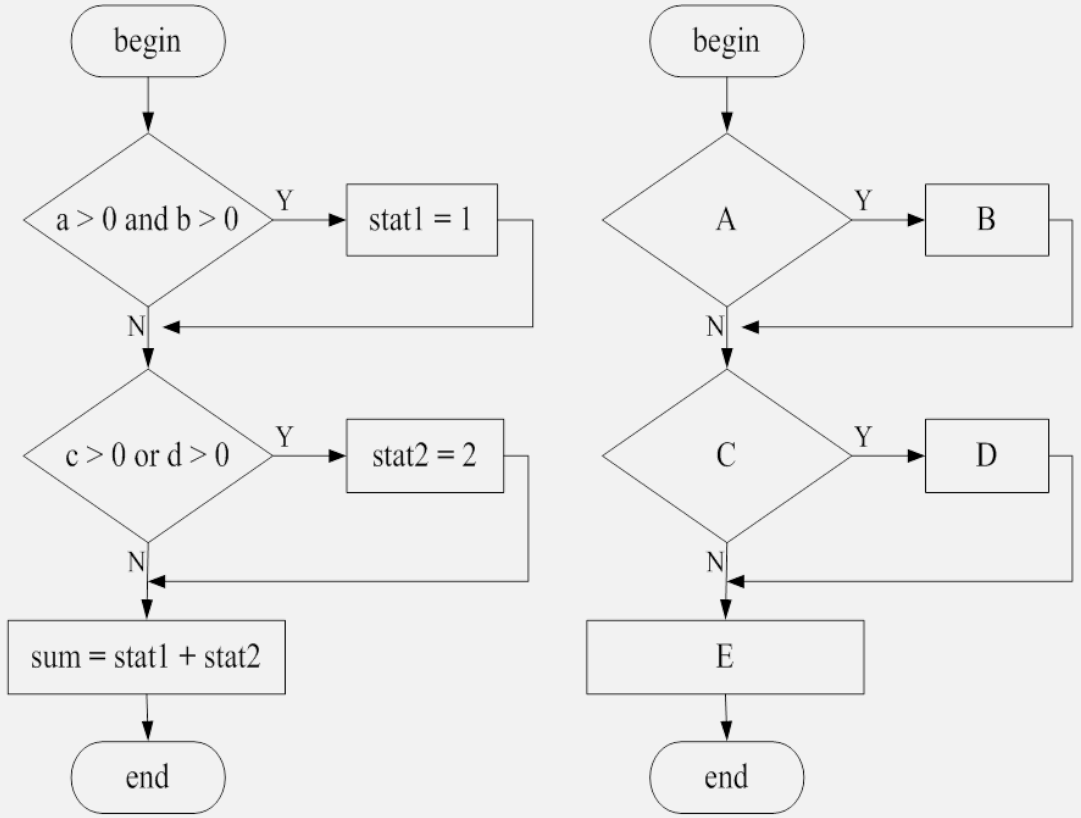


- ✓ 语句覆盖
- ✓ 分支（判定）覆盖
- ✓ 组合覆盖
- ✓ 基本路径覆盖

白盒测试方法

为了通俗易懂地说明这四种覆盖技术，请看如下案例。

【例】假定测试人员通过阅读分析其软件模块的代码后，得出如下左图的**程序流程图**。为了说明覆盖技术，右图给出了对等的**语句块简化流程图**。



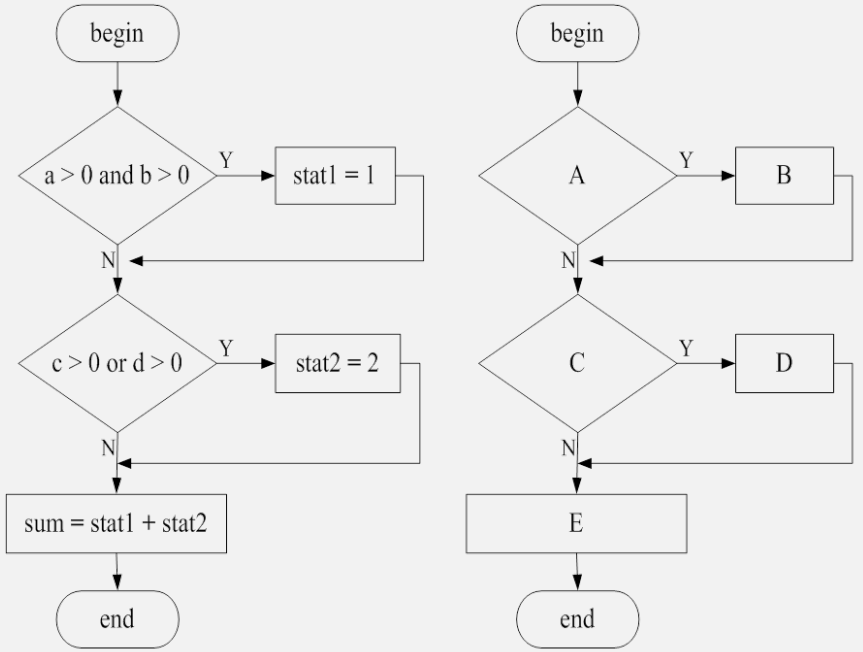
语句覆盖

- ✓ **语句覆盖**是最基本的覆盖，它要求设计足够多的测试用例，使得程序中**每条语句至少被执行一次**
- ✓ 若达到最高语句覆盖率，只要设计的测试用例能够**保证ABCDE代码块全都被执行一遍即可**，如下表所示。其中XX-V1-103-001代表一种测试用例的命名方式，它用“项目编号—项目版本号—模块号—测试用例编号”来表示用例序号。

测试用例序号 (项目编号—项目版本号—模块号—测试用例编号)	a取值	b取值	c取值	d取值	说明
XX-V1-103-001	1	1	1	1	覆盖ABCDE

语句覆盖

- ✓ 从表中可以看出**一个测试用例**就可以**使语句覆盖率达到100%**。
- ✓ 我们是否可以认为，白盒测试的任务完成了呢?
- ✓ **不可以**，C判定中的两个条件“是/或”的关系，设计测试用例时只要满足判定为真即可，那么如果c的值取真，d的值可以是真也可以是假，假如d为零会引起系统异常，那么d取值的任意性就容易忽略这种情况
- ✓ 100%的**语句覆盖率**也**不能将语句块**进行彻底测试。



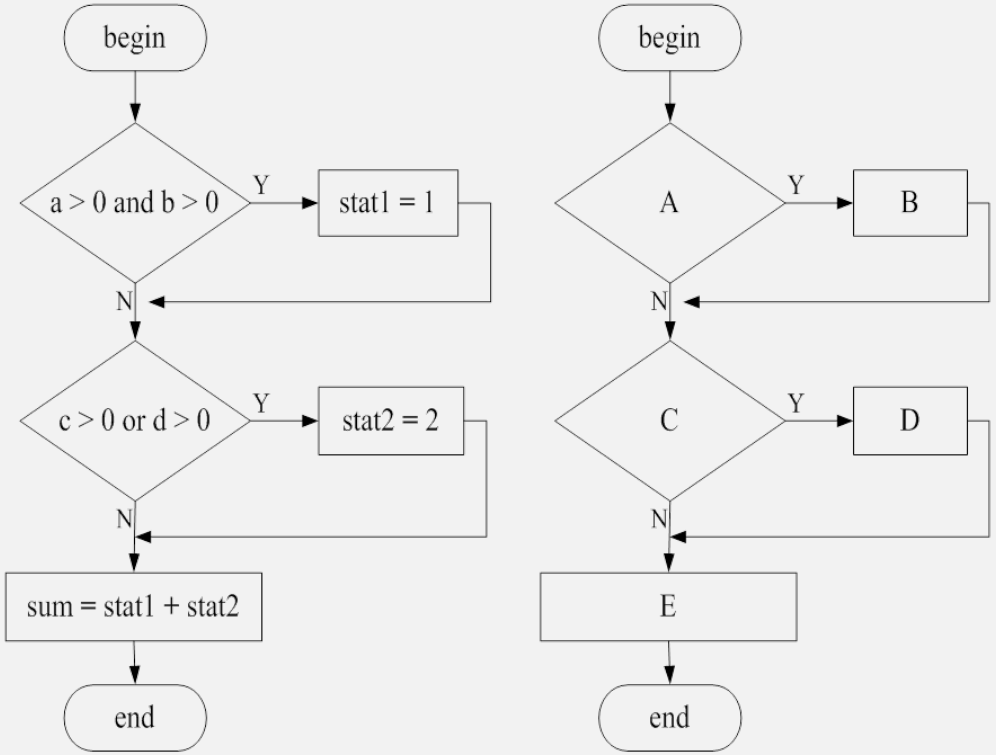
判定覆盖

- ✓ **判定覆盖又称为分支覆盖，它要求设计足够多的测试用例，使得程序中每个判定至少取一次真值和一次假值。**
- ✓ **为了达到最高判定覆盖率，我们需要设计测试用例，使得判定块A取一次真值和一次假值，判定块C取一次真值和一次假值，如下表所示：**

测试用例序号	a取值	b取值	c取值	d取值	说明
XX-V1-103-001	1	1	1	1	判定A为真，判定C为真，覆盖ABCDE
XX-V1-103-002	0	0	0	0	判定A为假，判定C为假，覆盖ACE

判定覆盖

- ✓ 表中的两个测试用例使得判定覆盖率达到100%，其中用例XX-V1-103-001使得语句覆盖率也达到100%。
- ✓ 此时，我们是否可以认为，白盒测试的任务完成了呢?
- ✓ 判定覆盖在**语句覆盖的基础上，增加了对判定分支的覆盖**，特别适合发现程序中只处理正常情况，而**忽略异常情况这类缺陷。**



组合覆盖

✓ 要求设计足够多的测试用例，使得每个判定中条件结果的**所有组合至少出现一次**。如下表所示：

测试用例序号	a取值	b取值	c取值	d取值	说明
XX-V1-103-001	1	1	1	1	A为真，C为真，覆盖ABCDE
XX-V1-103-002	1	1	1	0	A为真，C为真，覆盖ABCDE
XX-V1-103-003	1	1	0	1	A为真，C为真，覆盖ABCDE
XX-V1-103-004	1	1	0	0	A为真，C为假，覆盖ABCE
XX-V1-103-005	1	0	1	1	A为假，C为真，覆盖ACDE
XX-V1-103-006	1	0	1	0	A为假，C为真，覆盖ACDE
XX-V1-103-0012	1	0	0	1	A为假，C为真，覆盖ACDE
XX-V1-103-008	1	0	0	0	A为假，C为假，覆盖ACE

组合覆盖

XX-V1-103-009	0	1	1	1	A为假，C为真，覆盖ACDE
XX-V1-103-010	0	1	1	0	A为假，C为真，覆盖ACDE
XX-V1-103-011	0	1	0	1	A为假，C为真，覆盖ACDE
XX-V1-103-012	0	1	0	0	A为假，C为假，覆盖ACE
XX-V1-103-013	0	0	1	1	A为假，C为真，覆盖ACDE
XX-V1-103-014	0	0	1	0	A为假，C为真，覆盖ACDE
XX-V1-103-015	0	0	0	1	A为假，C为真，覆盖ACDE
XX-V1-103-016	0	0	0	0	A为假，C为假，覆盖ACE

基本路径测试的思想

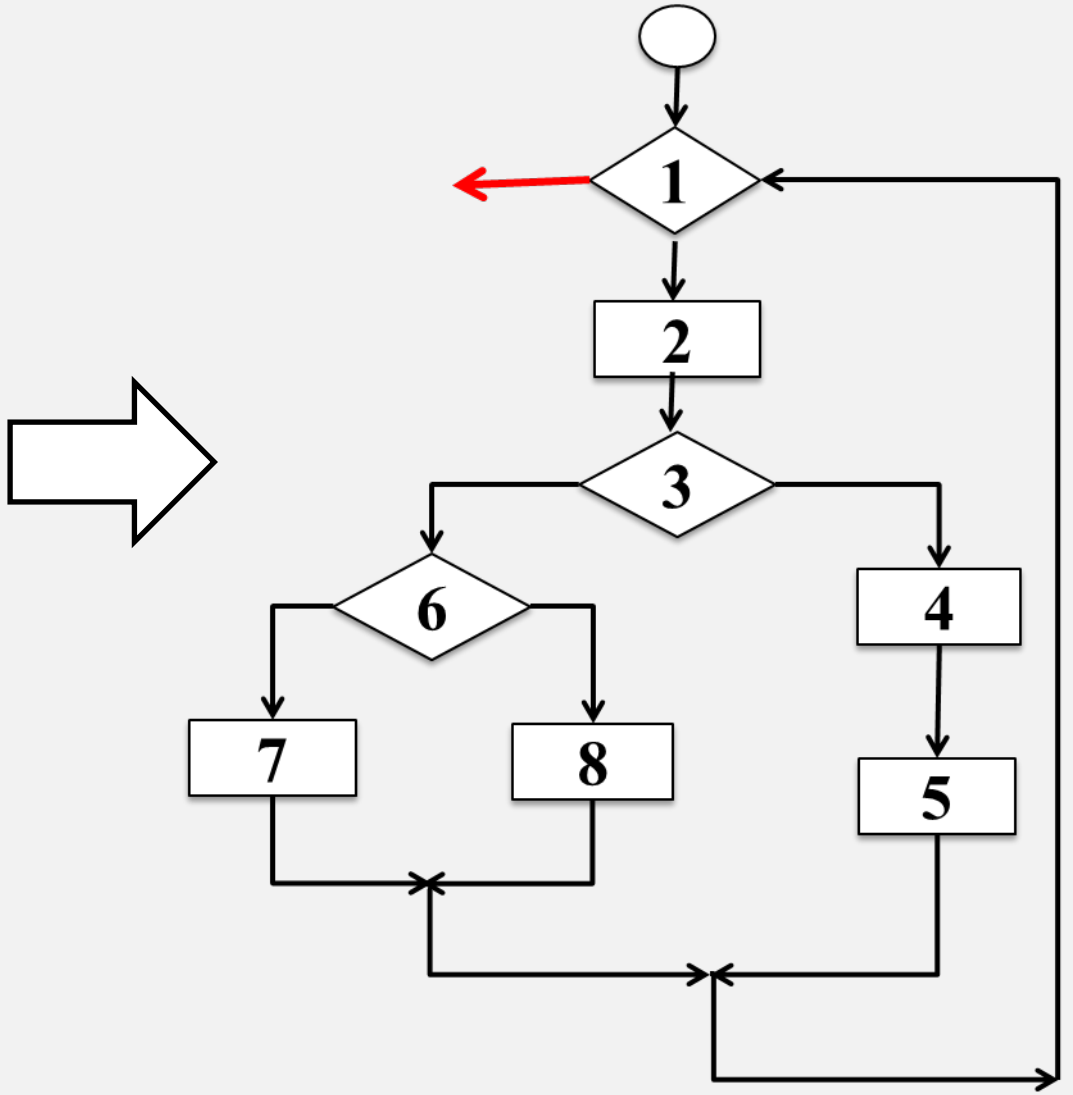
- 思想
  - 路径 ➔ 基本路径 ➔ **基本路径测试**
- 基本路径
  - 至少**引入一个新语句或者新判断**的程序通道
- 前提
  - 软件模块的逻辑结构（流程图）
  - 据此可掌握模块的基本路径
- 如何设计测试用例实现基本路径覆盖
  - 哪些基本路径 ➔ **流图** ➔ **流程图**

基本路径测试可实现基本路径的测试覆盖

步骤1：根据程序逻辑画出流程图

```
void Func(int nPosX, int nPosY) {
    while (nPosX > 0) {
        int nSum = nPosX + nPosY;
        if (nSum > 1) {
            nPosX减一;
            nPosY减一;
        }
        else {
            if (nSum < -1) nPosX -= 2;
            else nPosX减四;
        }
    }    // end of while
}
```

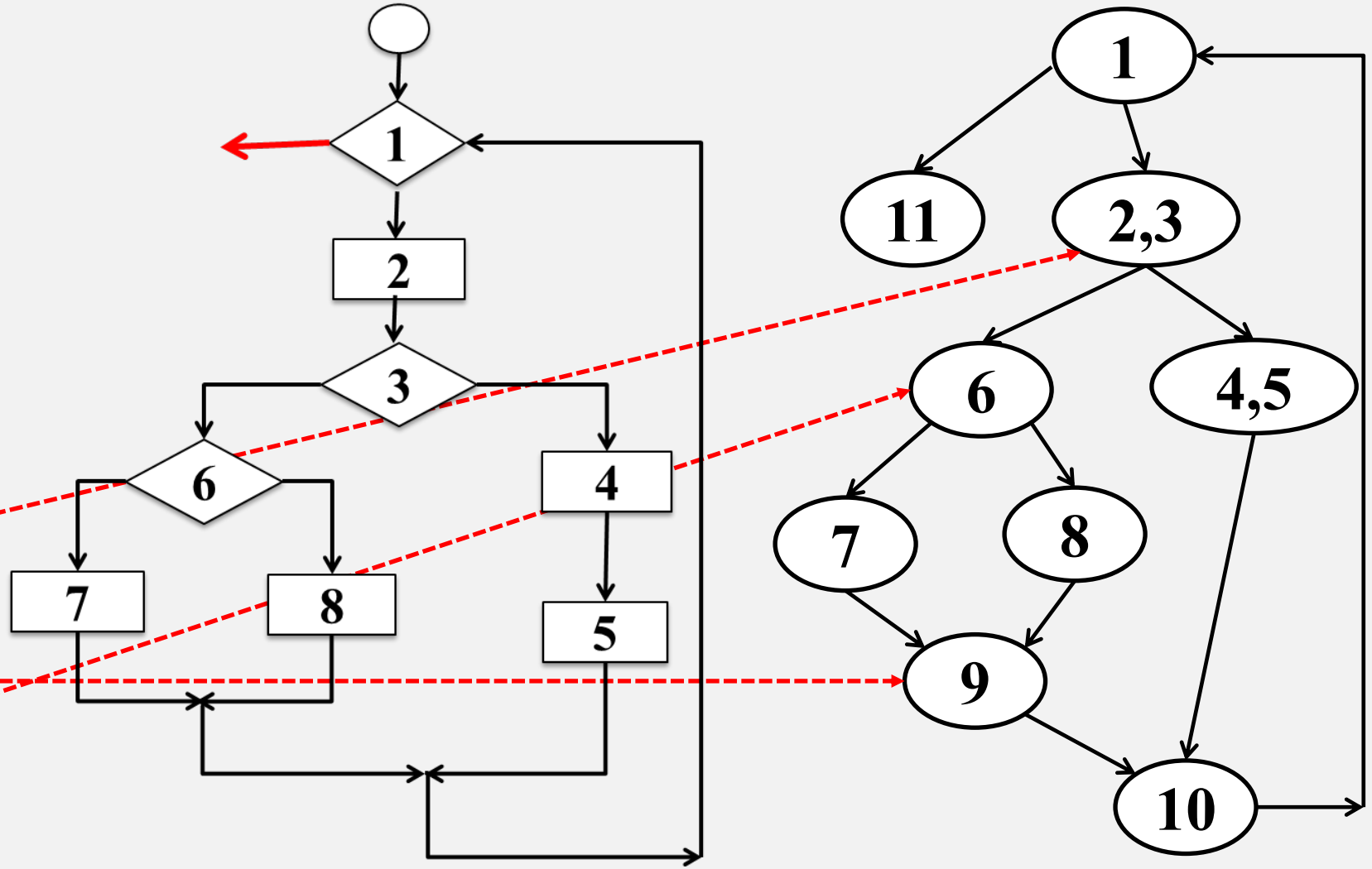
软件模块设计细节



注意流程图的正确画法！

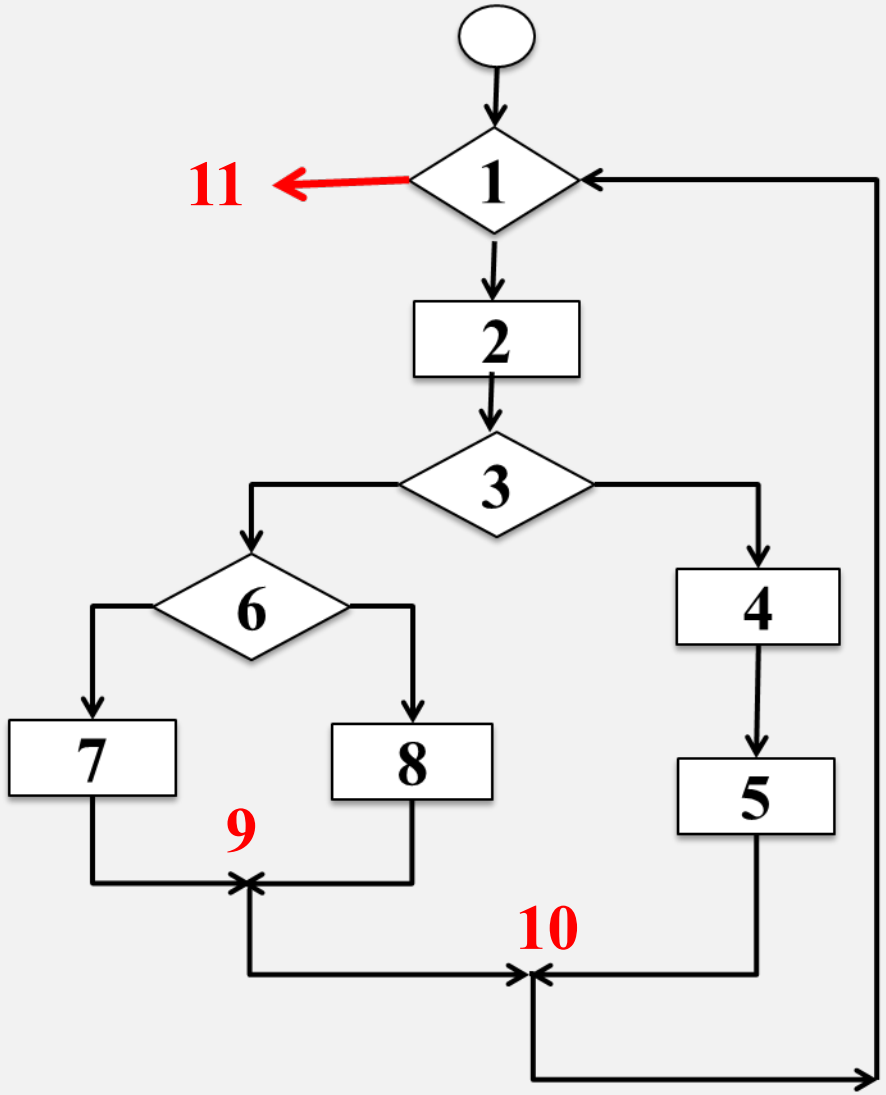
步骤2: 将流程图转换为流图 (1/2)

- 何为流图
  - 流图刻画程序控制结构但不涉及程序过程性细节
- 节点形式
  - **过程块**
  - **结合点**
  - **判定点**

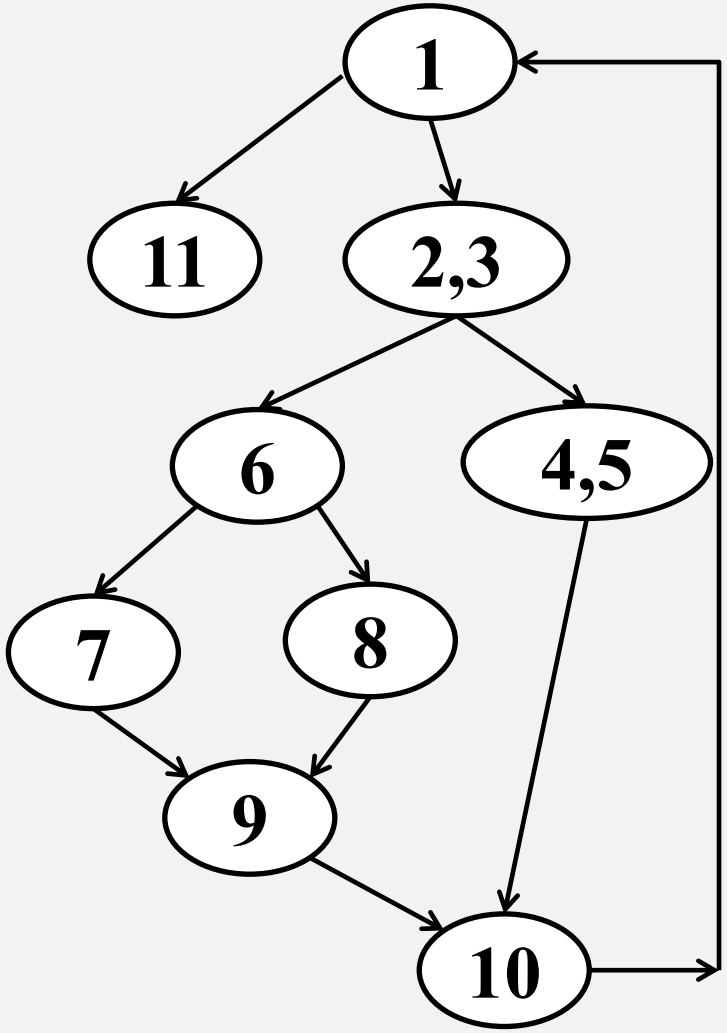


注意流图正确画法!

步骤2: 将流程图转换为流图 (2/2)



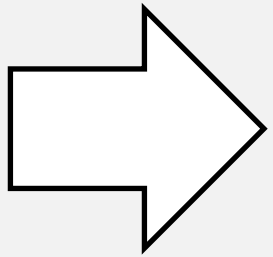
判定点
过程块
结合点

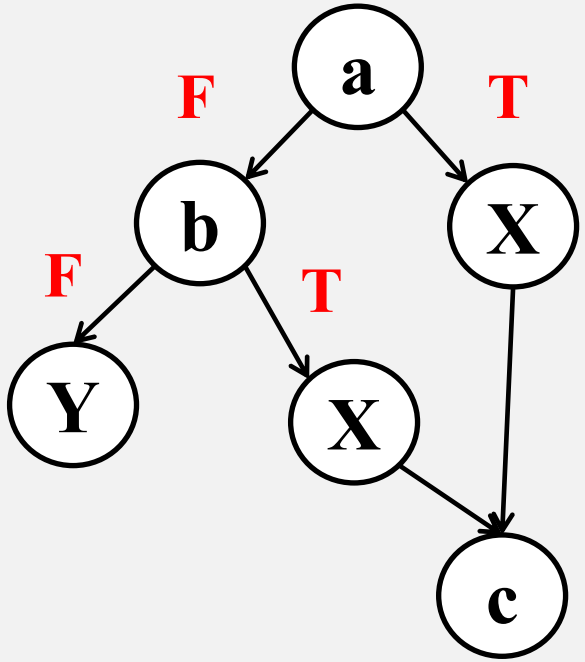


流图中的判定点不应含复合条件

- 否则按下列方式增加判定点

```
If a or b
Then X
Else  Y
End If
```



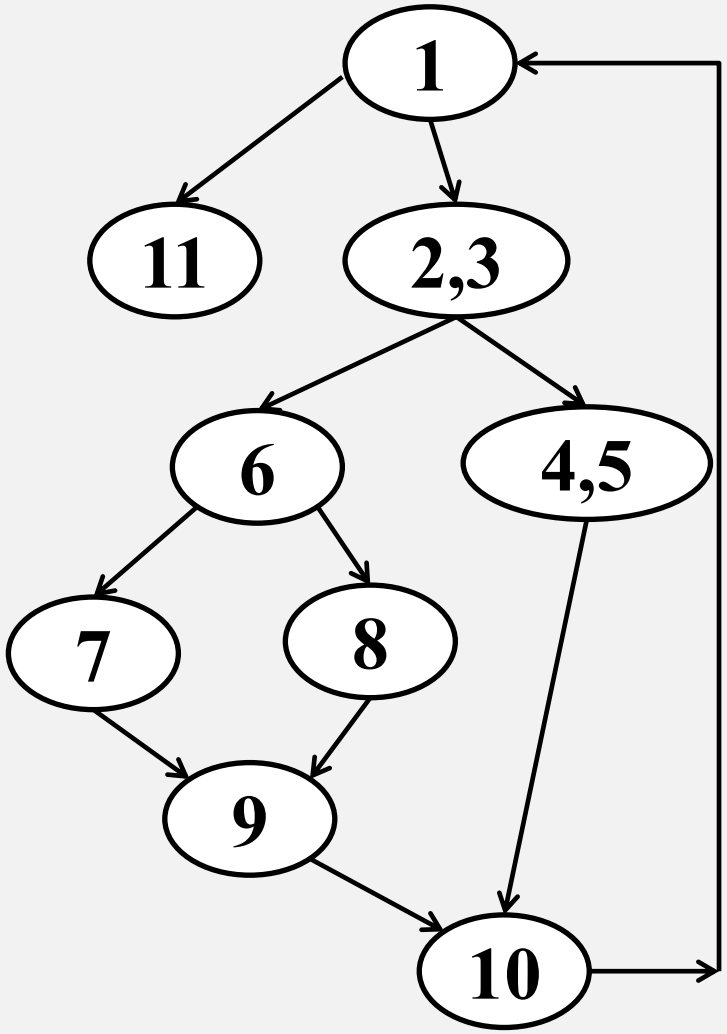


步骤3: 确定基本路径集合

- 基本路径的数量
  - 流图**Cyclomatic复杂度**
  - V(G) = E(dges) – N(odes) + 2
  - V(G) = 11 - 9 + 2 = 4

① 1 - 11
② 1 - 2, 3 - 6 - 7 - 9 - 10 - 1 - 11
③ 1 - 2, 3 - 4, 5 - 10 - 1 - 11
④ 1 - 2, 3 - 6 - 8 - 9 - 10 - 1 - 11

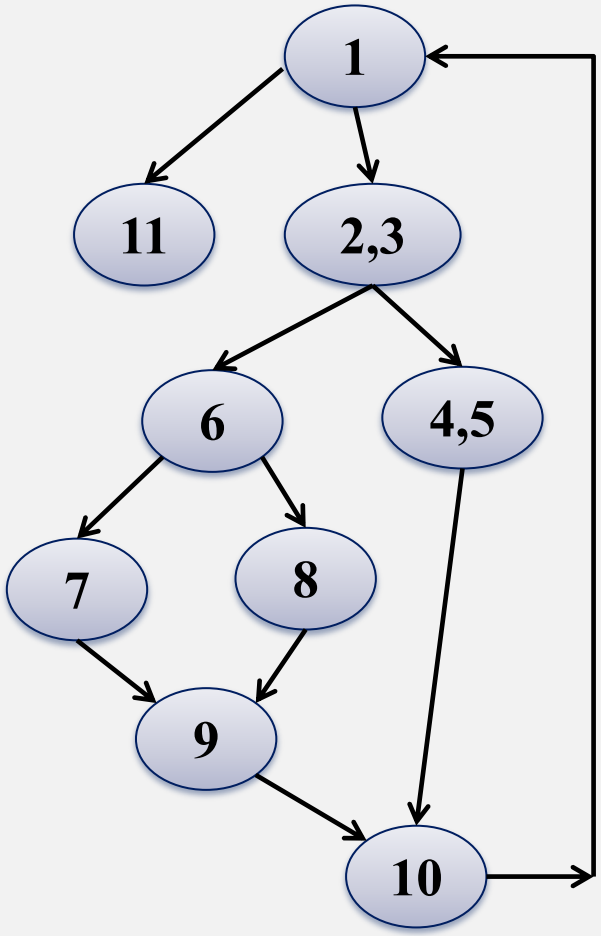
4条基本路径



步骤4: 针对测试路径设计测试用例

- 1-11
  - nPosX 取-1, nPosY取任意值
- 1 - 2, 3 - 4, 5 - 10 - 1 - 11
  - nPosX 取1, nPosY取1
- 1- 2, 3- 6-7 - 9 - 10-1– 11
  - nPosX 取1, nPosY取-1
- 1- 2, 3 - 6 - 8 - 9 - 10-1-11
  - nPosX 取1, nPosY取-3

```
void Func(int nPosX, int nPosY) {
    while (nPosX > 0) {
            int nSum = nPosX + nPosY;
            if (nSum > 1) {
                nPosX减一;
                nPosY减一;
            }
            else {
                if (nSum < -1) nPosX -= 2;
                else nPosX减四;
            }
    }       // end of while
}
```



描述每一个测试用例

测试用例标识:	对该测试用例赋予一个唯一标识
测试用例开发者:	谁编写的本用例
测试用例开发日期:	编写用例的日期
测试项:	描述将被测试的具体特征、代码模块等对象
测试输入:	测试时为程序提供的输入数据
前提条件:	执行测试时系统应处于的状态或要满足的条件等
环境要求:	执行测试所需的软硬件环境、测试工具、人员等
测试步骤:	(1)…; (如点击"文件"菜单中的"新建"菜单项) (2)…; (如在"test case"目录下选择"test5.dat"文件) ……
预期输出:	希望程序运行得到的结果
用例间的依赖性:	该测试用例依赖或受影响的其它测试用例

步骤5: 运行程序检验测试用例

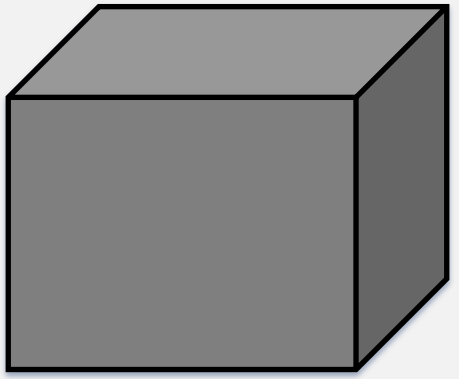
- 运行待测试的程序代码
- 逐个输入测试用例
- 分析程序的运行路径
- 如果运行路径与期望路径不一样，则存在缺陷

Part 2 软件测试分类

- 软件测试分类概述
- 软件白盒测试
- 软件黑盒测试

黑盒测试

- 思想
  - 根据已知的**程序功能和性能**(而非内部细节)，设计测试用例并通过测试检验程序的每个**功能和性能是否**正常
- 依据
  - 程序的**功能和性能描述**
- 特点
  - 知道程序功能和性能，**不必了解程序内部结构和处理细节**



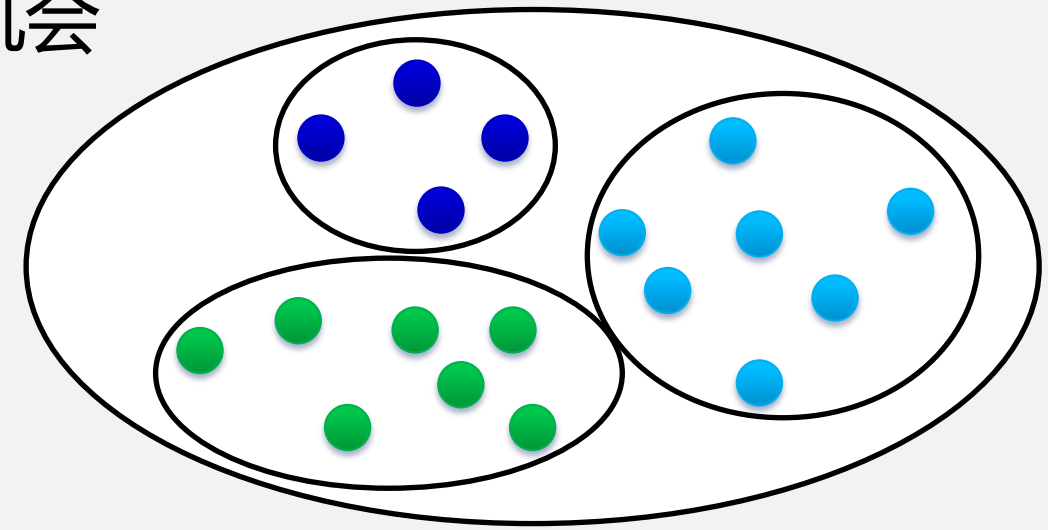
模块外在功能

1. 等价类划分法；
2. 边界值分析法；
3. 错误推测法；
4. 因果图分析法；
5. 场景分析法。

黑盒测试-等价分类法

- 思想
  - 把程序的输入数据集合**按输入条件**划分为若干个**等价类**
  - 每一个等价类对于输入条件而言为**一组有效或无效的输入**
  - 为**每一个等价类设计一个测试用例**
- 优点
  - 减少测试次数，不丢失发现错误的机会

每个等价类中的数据具有相同的测试特征



等价类划分法

有效等价类和无效等价类

- ✓ 每一个等价类又可以分为**有效等价类**和**无效等价类**。
- ✓ **有效等价类**，是指对于软件的需求规格说明书来说是**合理的、有意义**的输入数据构成的集合。利用有效等价类，可**检验软件是否实现了需求规格说明书**中所规定的需求。
- ✓ **无效等价类**主要是验证程序是否对**非法输入数据**进行处理。
- ✓ 黑盒测试法方法设计测试用例时，任务之一是**划分等价类**，区分两种等价类别，任务之二是在**每个等价类中，选取一个代表数据**作为测试用例。

等价类划分法

划分等价类并设计测试用例

①．在输入条件规定了**取值范围**时，可以确立**一个有效等价类**和**两个无效等价类**。

【例】如果输入值的范围是从1到999，则一个有效等价类为1<= 输入值<=999，两个无效等价类为小于1和大于999。如果是手工录入数据，还要考虑不输入数据，或者输入非法数据（例如非数值）等特殊情况。

②．当输入条件规定了**具体数值**，例如个数（或次数）时，可以**确立一个有效等价类，两个无效等价类**。

【例】用户登录时连续输入错误次数达到3次，则锁定账号。有效等价类为连续3次输入错误，两个无效等价类为连续输入错误次数小于3次和大于3次。

等价类划分法

③．当输入条件是**一个布尔量**，且限制为单选方式时，**可确定两个有效等价类**。

【例】网络购书时，选择开发票或者不开发票，就可以确定两个等价类；如果是手工输入，例如输入一个人的性别，则可以把男性和女性作为两个等价类，不输入数据和输入非法数据（非男非女）等也是常见的无效等价类。

④．当输入条件是**一组数据-枚举类型**（假设有N个），且限制是以单选方式输入时，**可确定N个有效等价类**。

【例】用户的注册信息中，学历的选择包括初中、高中、专科、本科等，可以确定有N个学历有效等价类，同样，如果是手工输入数据，则还应该包含不输入数据、输入非学历数据等无效等价类。

等价类划分法

⑤．当输入条件是一组数据（假设是N个），并且可以允许多选方式输入时，则除了将每个选项作为一个单独的有效等价类外，还应补充选任意两个，选任意N个，作为有效等价类。

【例】用户选择自己的兴趣爱好类别，音乐、体育、绘画等。则

有效等价类为所有选项的组合。

⑥．在规定了输入数据必须遵守的规则的情况下，可确立若干个有效等价类（符合规则）和若干个无效等价类（从不同角度违反规则）。

【例】用户名必须是由26个大小写英文字母、数字和下划线组成，长度为6到16位。则有效等价类为符合规则的用户名，还包括不符合规则的若干无效等价类，例如长度不满足规则，包含的字符不满足规则，不输入任何字符等。

实例 - 等价类划分法

- **三角形问题：接受三个整数a, b, c作为输入，用作三角形的边，程序输出由这三个边确定的三角形的类型：等边三角形、等腰三角形、不等边三角形、非三角形。**

输入条件	有效等价类	无效等价类
是否三角形的三个边	(a>0) (1) (b>0) (2) (c>0) (3) (a+b>c) (4) (b+c>a) (5) (a+c>b) (6)	(a<=0) (7) (b<=0) (8) (c<=0) (9) (a+b<=c) (10) (b+c<=a) (11) (a+c<=b) (12)
是否等腰三角形	(a=b) (13) (b=c) (14) (a=c) (15)	(a!=b)and(b!=c)and(a!=c)(16)
是否等边三角形	(a=b) and (b=c) and (a=c) (17)	(a!=b) (18) (b!=c) (19) (c!=a) (20)

实例 - 等价类划分法

根据等价类表，该问题的测试用例设计如下表所示。

序号	[a,b,c]	覆盖等价类	输出
1	3，4，5	(1)，(2)，(3)，(4)，(5)，(6)	不等边三角形
2	0，1，2	(7)	非三角形
3	1，0，2	(8)	
4	1，2，0	(9)	
5	1，2，3	(10)	
6	3，1，2	(11)	
12	1，3，2	(12)	
8	3，3，4	(1)，(2)，(3)，(4)，(5)，(6)，(13)	等腰三角形
9	3，4，4	(1)，(2)，(3)，(4)，(5)，(6)，(14)	
10	3，4，3	(1)，(2)，(3)，(4)，(5)，(6)，(15)	
11	3，4，5	(1)，(2)，(3)，(4)，(5)，(6)，(16)	非等腰三角形
12	3，3，3	(1)，(2)，(3)，(4)，(5)，(6)，(17)	等边三角形
13	3，4，4	(1)，(2)，(3)，(4)，(5)，(6)，(18)	非等边三角形
14	3，4，3	(1)，(2)，(3)，(4)，(5)，(6)，(19)	
15	3，3，4	(1)，(2)，(3)，(4)，(5)，(6)，(20)	

边界值分析法

基于边界值分析方法选择测试用例的原则

①．如果输入条件规定了值的范围，则应取刚刚达到这个范围的边界的值，以及刚刚超越这个范围的边界值作为测试输入数据。

②．如果输入条件规定了值的个数，则用最大个数、最小个数、比最小个数少1、比最大个数多1的数，作为测试数据。

③．根据规格说明的每个输入条件，使用前面的原则①和② 。

④．如果程序的规格说明给出的输入域或输出域是有序集合，则应选取集合的第一个元素和最后一个元素作为测试用例。

⑥．我们在确定边界值时，还要参考ASCII/Unicode编码表。

黑盒测试-边界值分析法

- 输入条件是一范围(a,b)
  - a,b以及紧挨a,b左右的值应作为测试用例
- 输入条件为一组数
  - 选择这组数最大者和最小者，次大和次小者作为测试用例
- 如果程序的内部数据结构是有界的
  - 应设计测试用例使它能够检查该数据结构的边界

边界值分析法示例

- z = func(x, y):
  - 当 0 < x < 1024 且 y = 0， z = -1
  - 否则， z = x * lg(y)
- 关于x的等价类6个
  - -1, 0, 1
  - 1023, 1024, 1025
- 关于y的等价类3个
  - -1, 0, 1

测试用例18个

① X=1,y=0, z=-1
② X=1,y=-1, z=**
③ X=1,y=1 , z=**
④ X=0,y=0 , z=**
⑤ X=0,y=-1 , z=**
⑥ X=0,y=1 , z=**
⑦ X=-1,y=0 , z=**
⑧ X=-1,y=-100 , z=**
⑨ X=-1,y=200 , z=**
⑩ ……

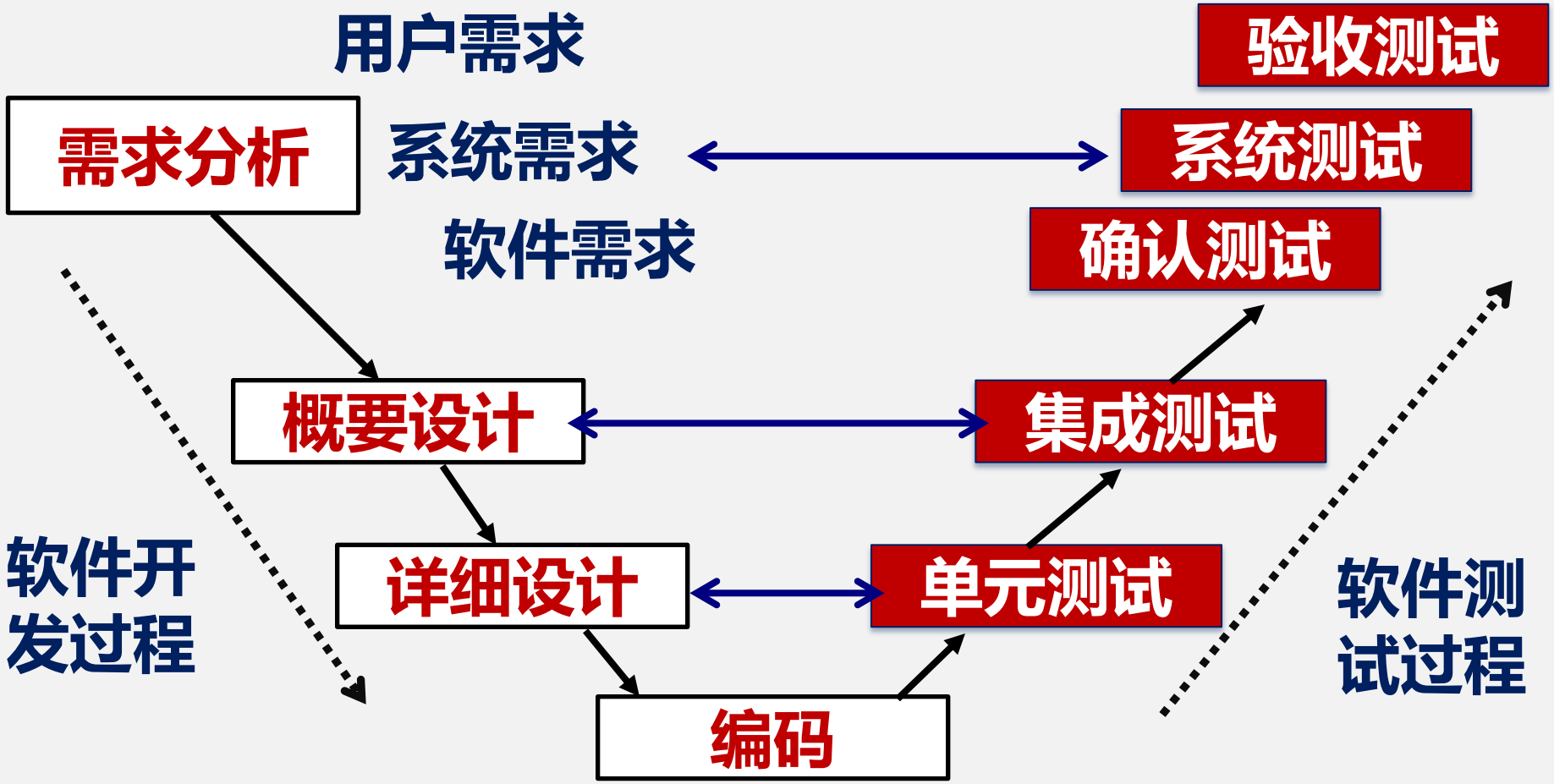
北京航空航天大學
BEIHANG UNIVERSITY



Part 3

软件测试过程

软件测试活动



要对软件进行系统的测试

单元测试

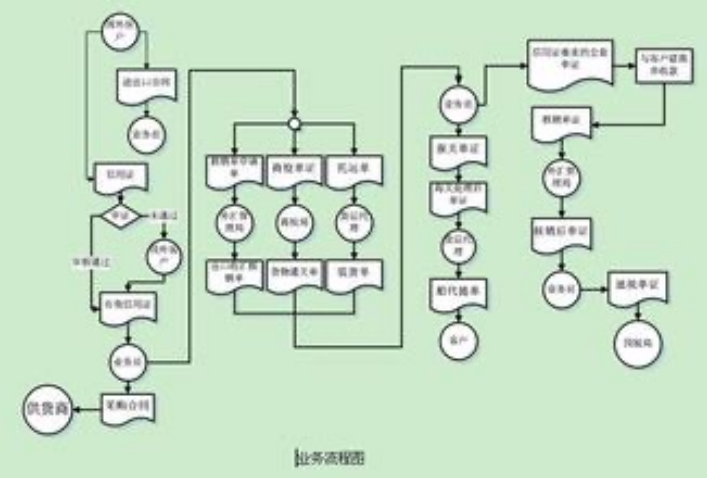
- **测试对象**
  - ✓对软件基本模块单元进行测试
  - ✓过程、函数、方法、类
- **测试方法**
  - ✓大多采用**白盒测试**技术
- **测试的内容**
  - ✓模块接口测试
  - ✓模块局部数据结构测试
  - ✓模块独立执行路径测试
  - ✓模块错误处理通道测试
  - ✓模块边界条件测试

程序单元是构成软件系统的基本要素，必须对其进行测试以发现缺陷

在详细设计阶段就可以设计单元测试用例及计划

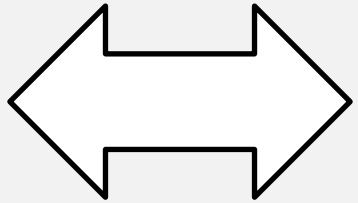
程序单元测试的依据和对象

程序模块功能、接口和流程



业务流程图

测试依据：设计文档



```java
package MarDetecter;

import java.awt.Image;

/*构建类：基础机器人；属性：当前位置；方法：在网格上由当前位置单步移动到另外一个位置*/
public class BasicRobot extends Agent {
    /* 基础机器人的位置*/
    public Coordinate posi = null;
    private BasicRobot basicrobot = null;

    public BasicRobot(){
        basicrobot = new BasicRobot();
    }
/*设置机器人位置*/
    public void setRobotPosi(Coordinate p){
        posi = p;
    }

/*获取机器人位置*/
    public Coordinate getRobotPosi(BasicRobot thisrobot){
        return(thisrobot.posi);
    }
    /* 基础机器人由当前位置单步移动到另外一个位置方法实现*/
    public void moveStep(Coordinate start, Coordinate next) {
        ShortestPath path = new ShortestPath();
        path.setStart(start.x, start.y);
        path.setTarget(next.x, next.y);

    }

    protected void setup() {
        System.out.println("BasciRobot'name is " + getLocalName());
        addBehaviour(new RandomWalkBev(this));

    }

}

```

测试对象：模块代码

谁负责开展程序单元测试?

- 单元测试必须由最熟悉代码的人来写
- 程序单元代码的编写者完成程序单元测试
- 程序员必须保证自己编写程序的质量

谁写的代码谁自己来负责单元测试

单元测试的结果是什么?

- 程序单元测试报告
  - 测试用例的设计
  - 程序单元的运行
  - 运行结果情况
  - 是否与预期相符一致
- 谁负责撰写该报告
  - 程序员



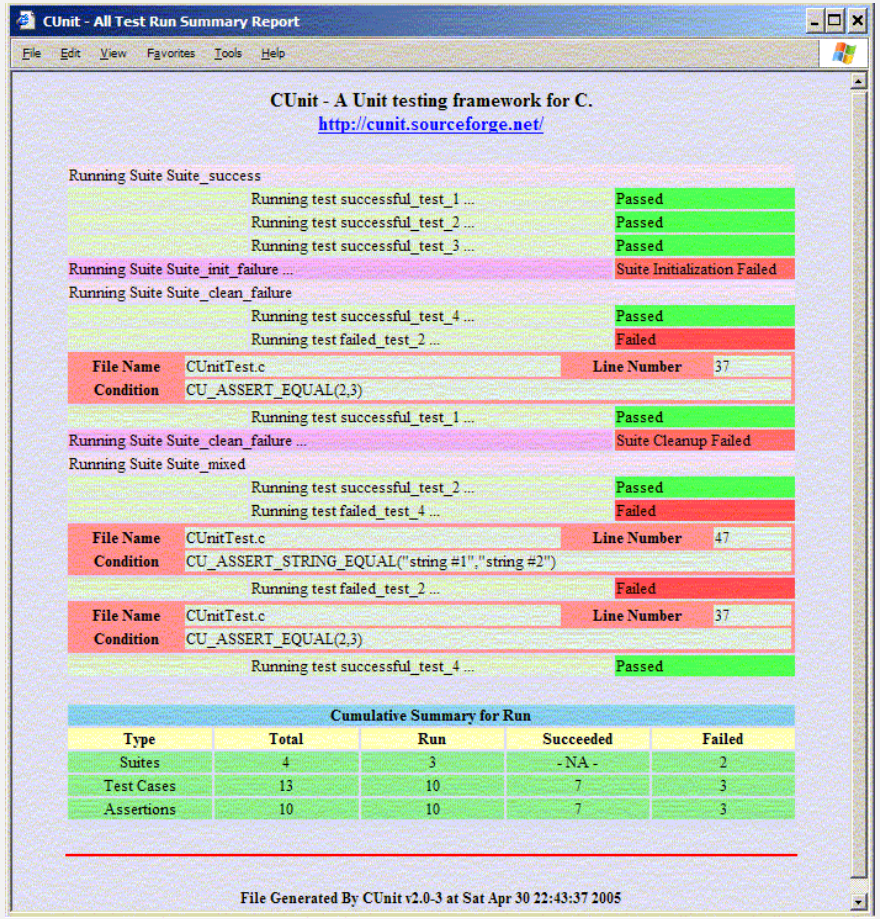
单元测试技术

- 测试设计：
  - 依据模块的内部结构，设计测试用例的过程。
  - 主要采用白盒测试技术，关注逻辑覆盖

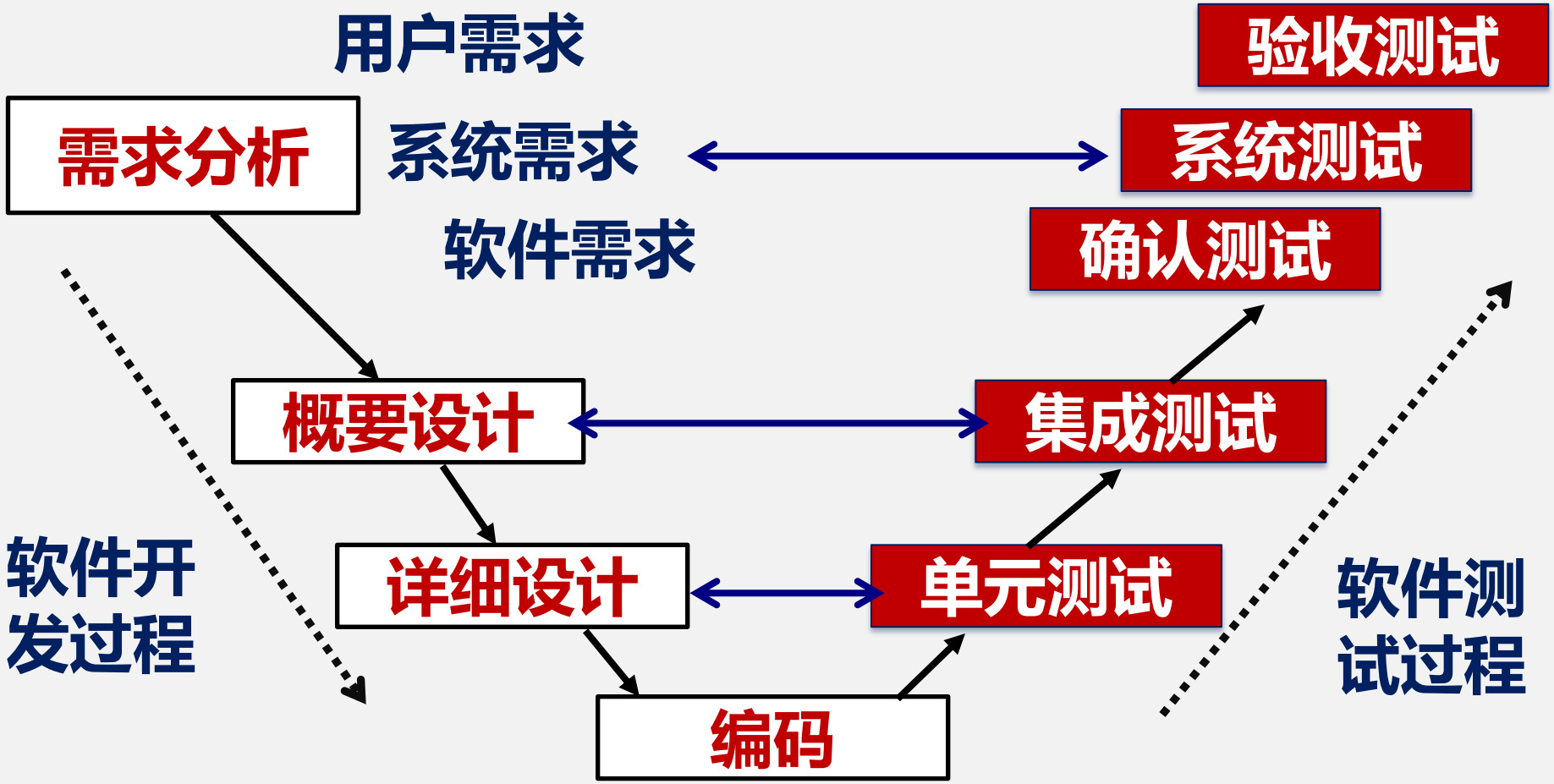
- 原则：
  - 保证没有死代码
    - 保证一个模块中的每个独立路径都可能被使用到
  - 保证对所有的逻辑值都测试true和false
  - 在上下边界和合法的范围内运行所有的循环
  - 确保内部数据结构的有效性和完整性

单元测试工具

- C/C++语言 - C++Test、Cunit、CppTest
- .Net开发 - Nunit
- Java语言 –Junit
- Python语言 - PyUnit

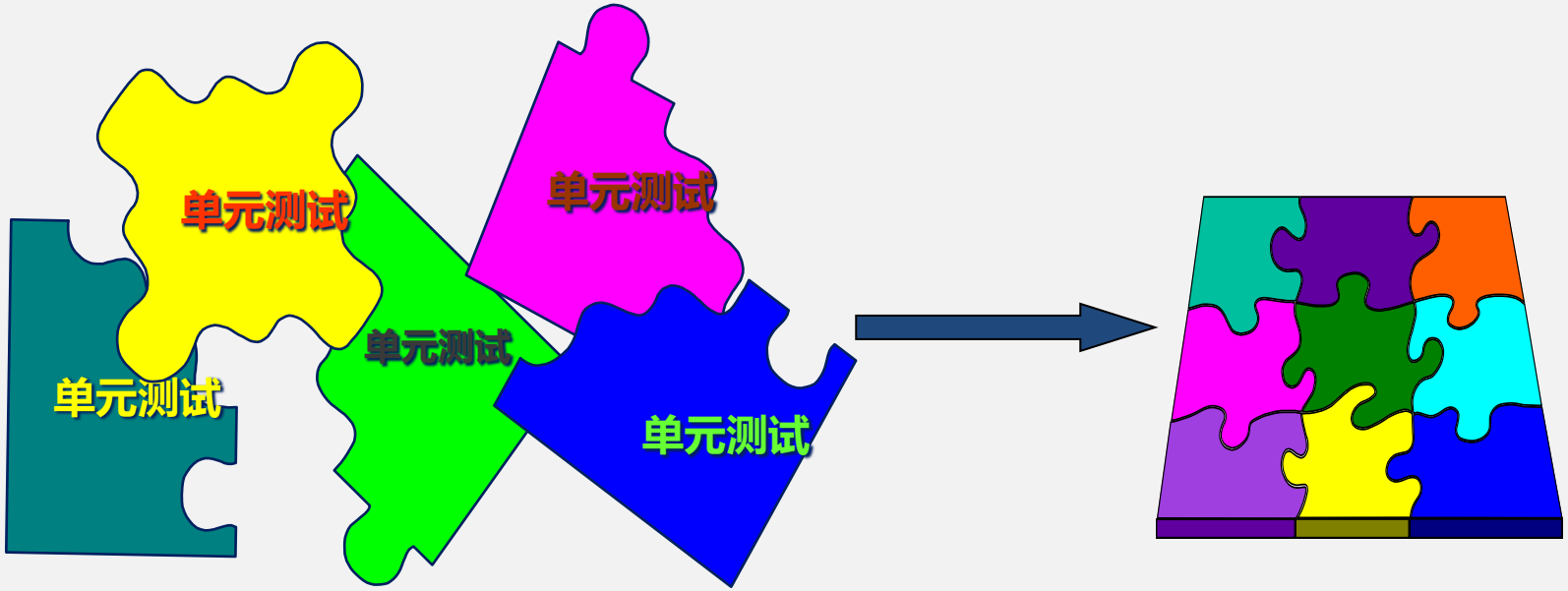


软件测试活动



要对软件进行系统的测试

集成测试



集成测试

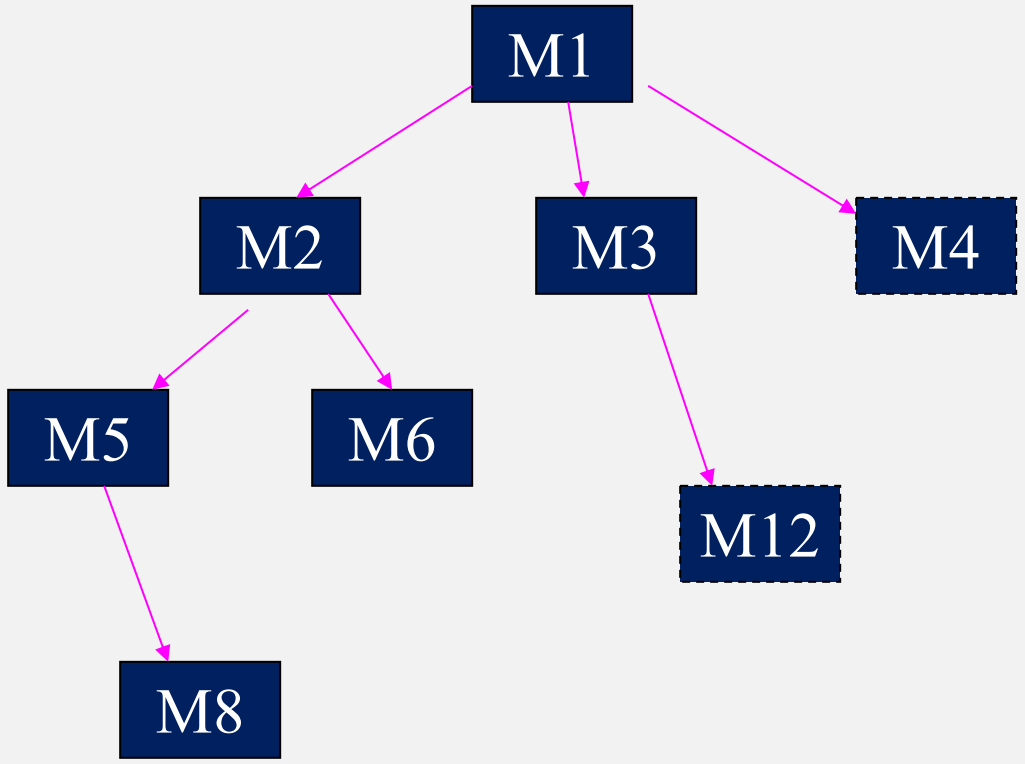
- **测试对象**
  - ✓对软件**模块之间的接口**进行测试
  - ✓过程调用、函数调用、消息传递、远程过程调用
- **测试技术**
  - ✓采用**黑盒测试**技术
- **集成测试的内容**
  - ✓过程调用
  - ✓函数调用
  - ✓消息传递
  - ✓远程过程调用
  - ✓网络消息



集成测试策略

- 非渐增式测试方法
  - 大爆炸集成
- 渐增式测试方法
  - 自顶向下集成
  - 自底向上集成
  - 三明治集成

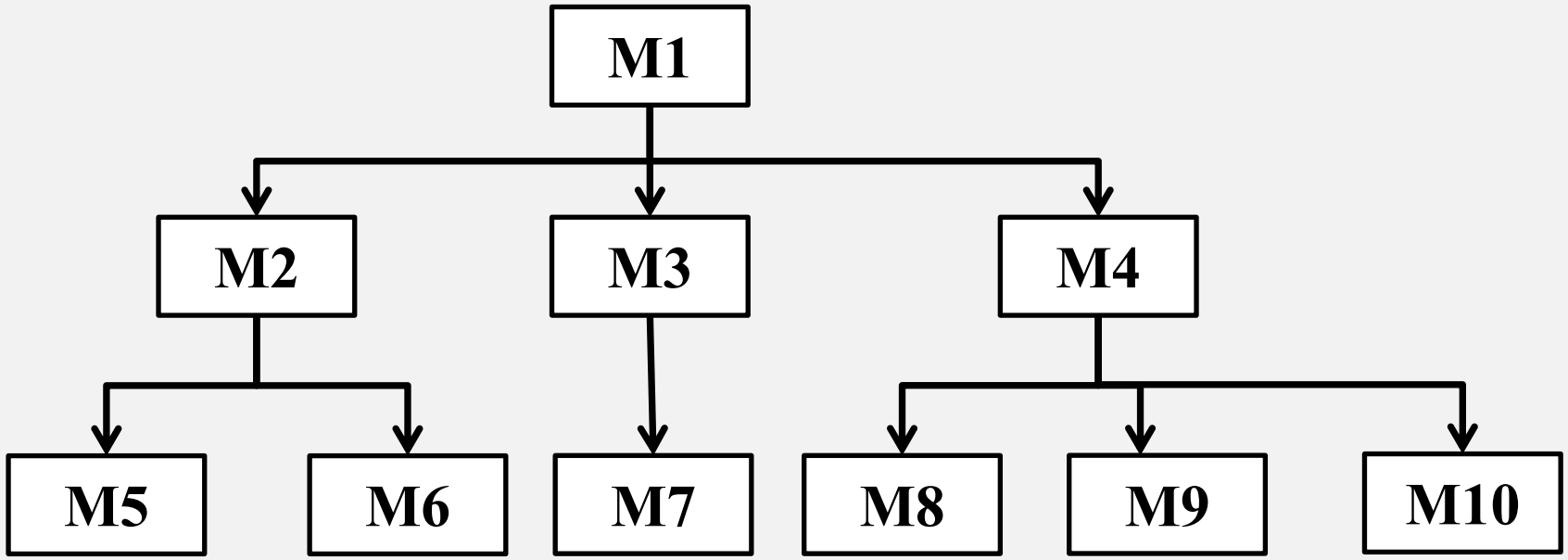
大爆炸集成



- 对所有模块进行单元测试后，按照程序结构图将各模块链接起来，把链接后的程序当作一个整体去进行测试

渐增式测试方法

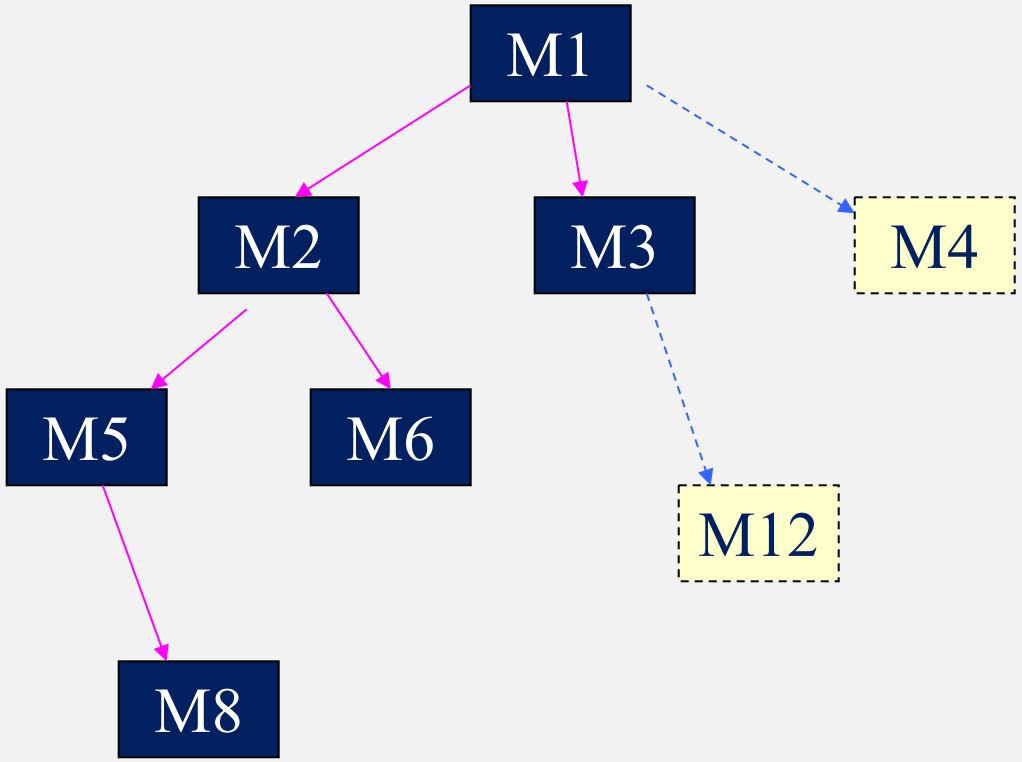
- **自顶向下集成**
  - ✓深度优先或者广度优先的策略把各个模块进行组装和测试
- **自底向上集成**
  - ✓自底向上进行组装和测试



在概要设计阶段就可以设计集成测试用例及计划

自顶向下集成

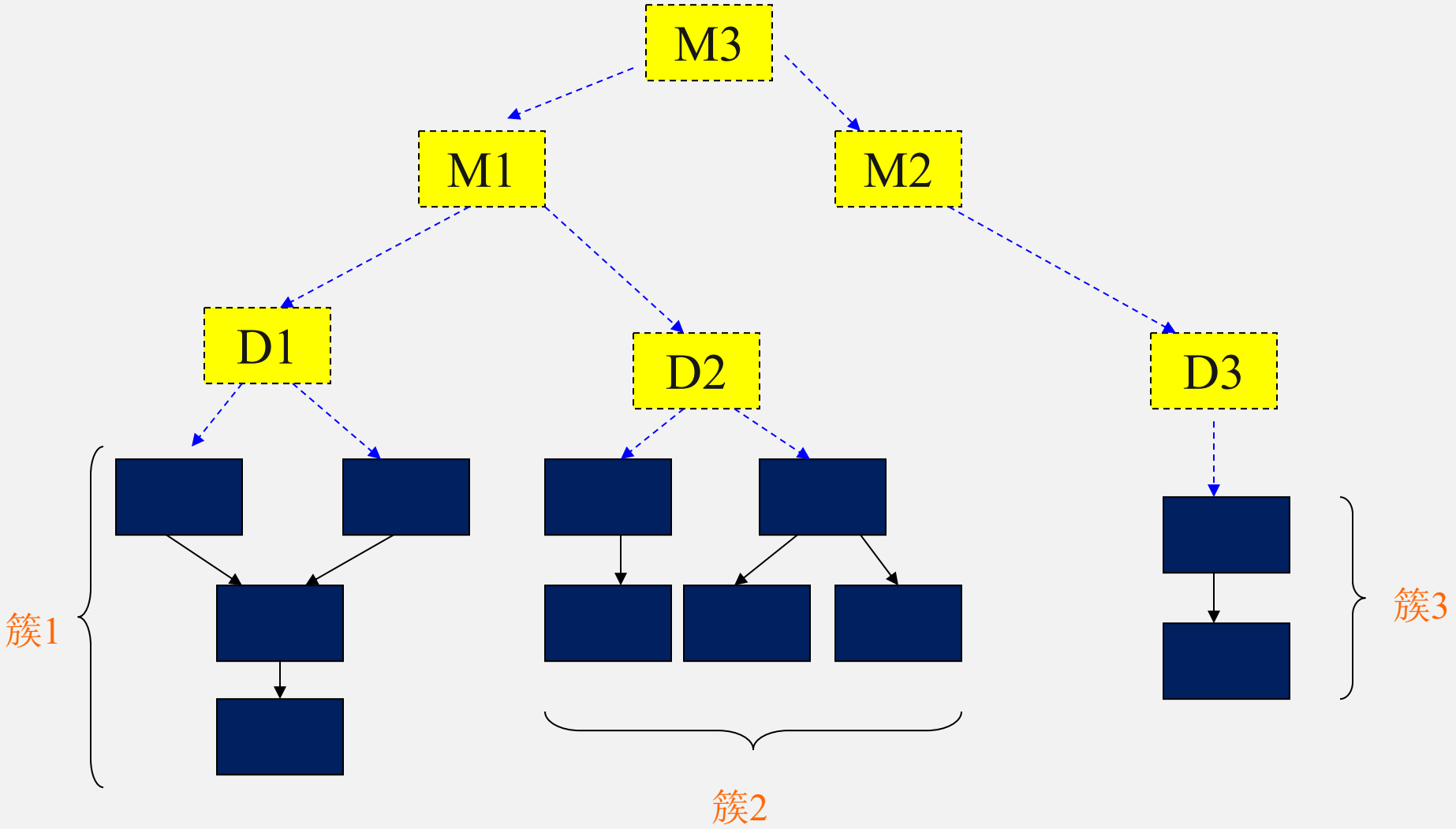
- 优点：只需要一个驱动模块、支持故障隔离



- 先集成主控模块，然后按照软件控制层次接口向下集成

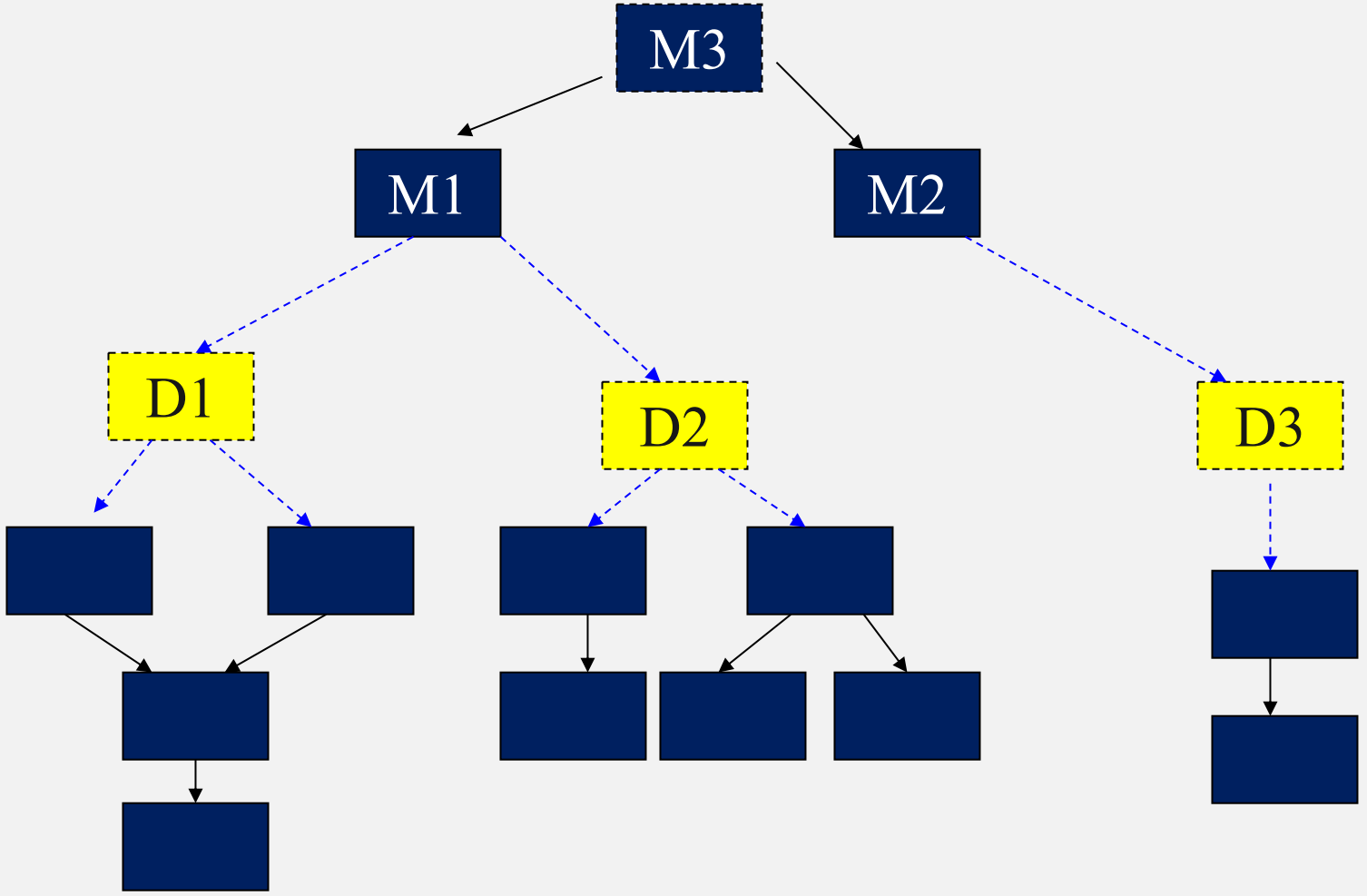
自底向上集成

- 优点：不用桩模块的开发，并行化高，减少回归测试成本



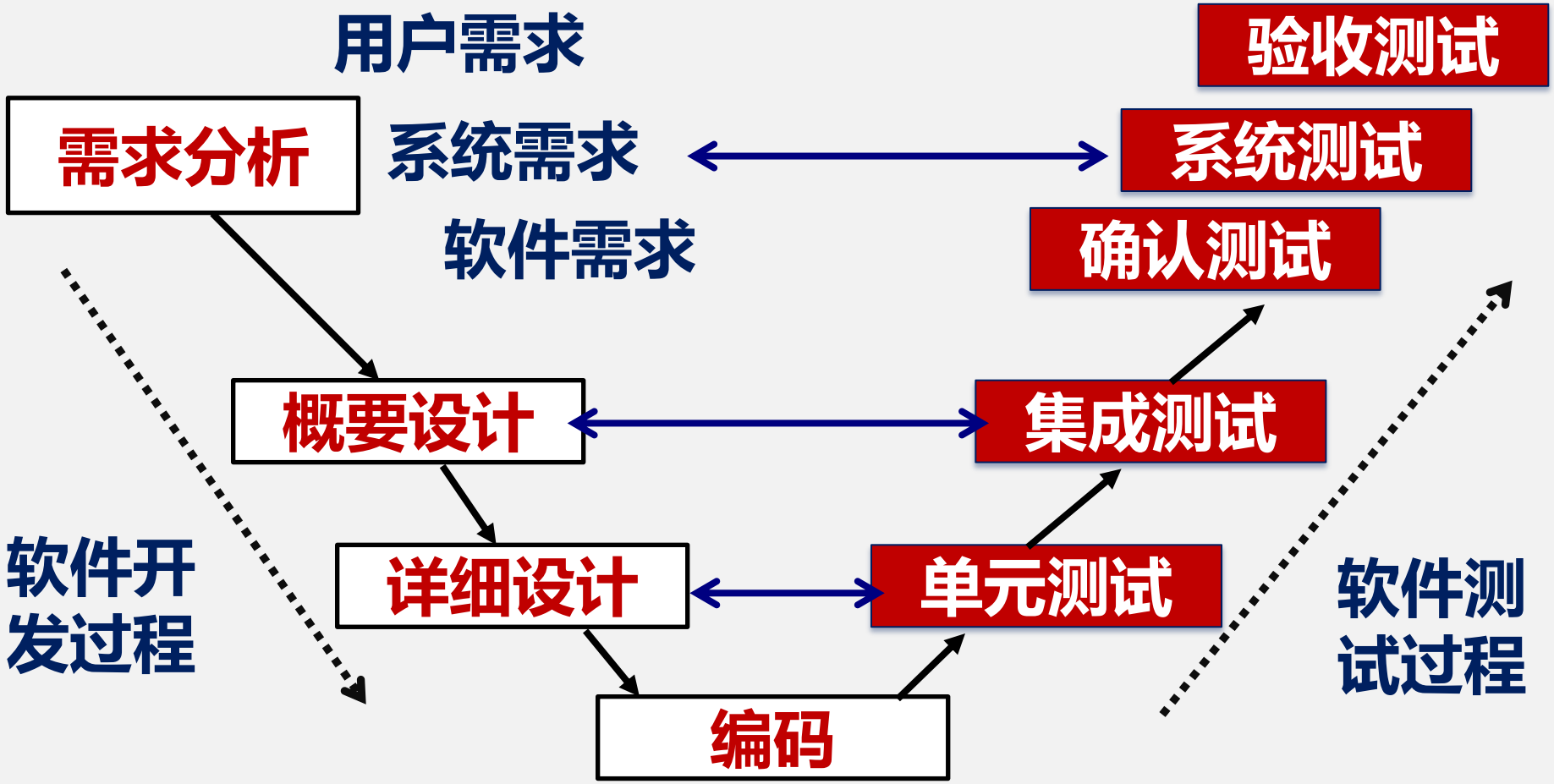
- 从最底层模块开始，逐步向上集成

三明治集成



- 同时进行自底向上与自顶向下

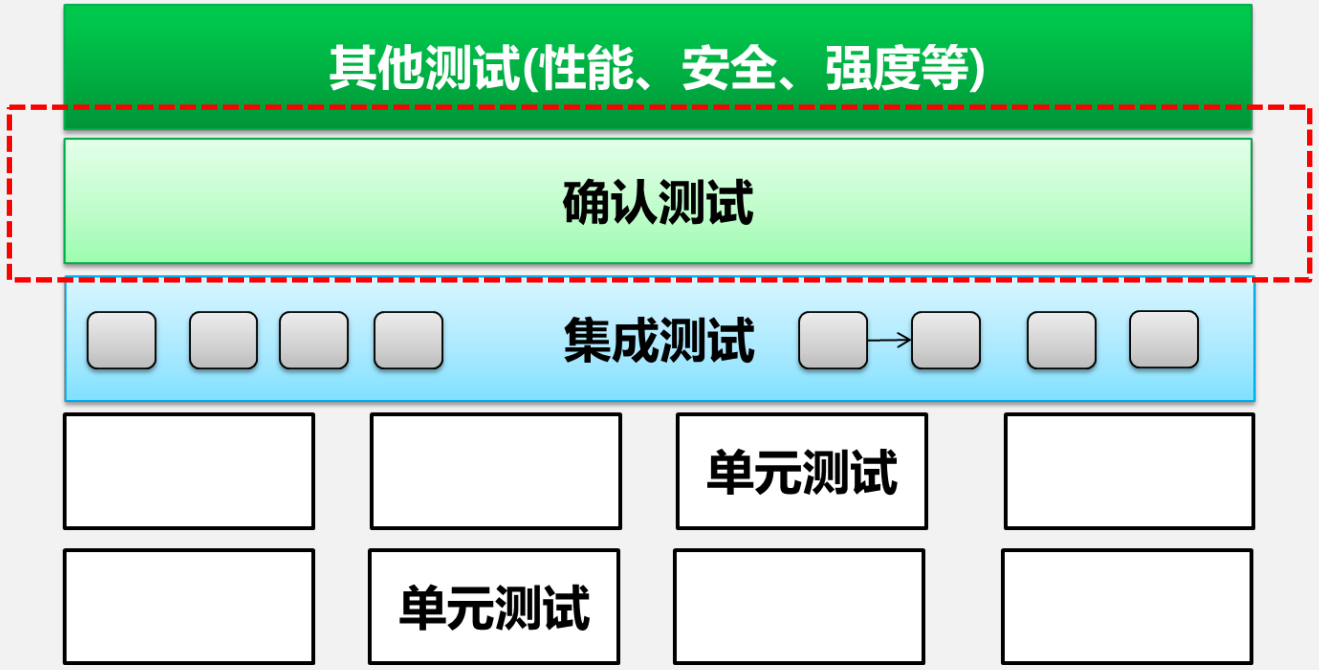
软件测试活动



要对软件进行系统的测试

确认测试

- **测试对象**
  - ✓对**软件**的功能和性能进行测试
  - ✓判断**目标软件系统**是否满足**软件需求**
- **依据和标准**
  - ✓软件需求规格说明书
- **测试技术**
  - ✓采用黑盒测试技术



在需求分析阶段就可以设计确认测试用例及计划

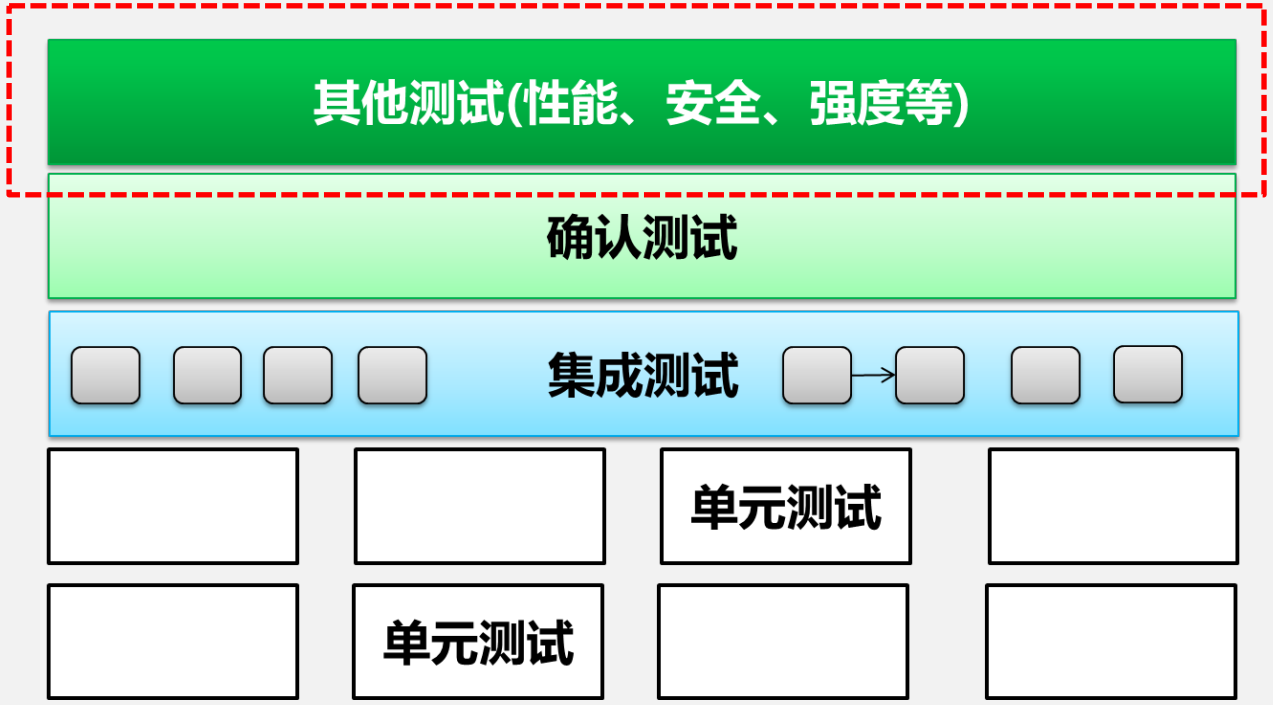
功能测试

功能测试

- ✓目标：对产品的功能进行测试，检验是否实现、是否正确实现
- ✓方法：覆盖产品的功能
  - 等价类划分
  - 边界值分析
  - 因果图
  - 判断表
  - 错误猜测法

非功能性测试

- 性能测试
- 强度测试
- 配置和兼容性测试
- 安全性测试
- 可靠性测试
- 用户界面测试
- 本地化测试
- Web测试
- 安装测试
- ……



在需求分析阶段就可以设计非功能测试用例及计划

性能测试

性能测试

- ✓目标：对产品的性能进行测试，检验是否达标、是否能够保持
- ✓方法：覆盖系统的性能需求，一般和负载测试结合使用
  - 口错误猜测法

性能测试用例示例

用例标识：	KCZC-XN-1
测试项：	课程注册管理系统的性能能否支持同时有100个用户登陆使用。并且平均响应时间少于1.5秒。
测试输入：	由模拟软件模拟100个用户使用系统。
前提条件：	软件系统已经完成并且在服务器上安装成功。
环境要求：	使用1台服务器，配置为××××； 使用5台微机作为客户机，配置为××××； 模拟软件在客户机安装；网络通畅。
测试步骤：	(1)启动服务器系统； (2)由模拟软件在每个客户机分别依次发送20个用户登陆请求，并模拟选课过程； (3)监控程序把每个用户的请求和服务器响应过程、时间记录在日志中； (4)对记录结果进行处理，判断是否每个用户均获得成功的服务，并计算平均响应时间。
预期输出：	每个模拟用户都得到正确服务，且对请求的平均响应时间小于1.5秒。

压力测试

压力测试

- ✓目标：在人为设置的系统资源紧缺情况下，检查系统是否发生功能或者性能上的问题
- ✓方法：人为减少可用的系统资源，包括：内存、硬盘、网络、CPU占用、数据库反应时间...
  - □等价类划分
  - □边界值分析
  - □错误猜测法

容量测试

容量测试

- ✓目标：在人为设置的高负载（大数据量、大访问量）的情况下，检查系统是否发生功能或者性能上的问题
- ✓方法：人为生成大数据量，并利用工具模拟频繁并发访问
  - □等价类划分
  - □边界值分析
  - □错误猜测法

安全性测试

- **设计测试用例来暴露软件安全漏洞的过程**
  - ✓如设法破坏数据库管理系统的数据安全机制、突破重要领域软件系统的用户访问控制等
- **安全性测试的过程**
  - ✓明确潜在的安全隐患，包括容易遭受攻击的内容、可能作为潜在入侵者的人员角色等
  - ✓明确潜在的入侵行为及时机，例如事务初始化、系统输入、执行存储和检索操作等时候
  - ✓列出每种潜在安全隐患可能遭到的入侵行为及其可能性
  - ✓确定风险较高的入侵点，这可通过对包括发生可能性、后果严重程度等因素进行分析得到
  - ✓根据风险的顺序，设计测试用例来实施安全性测试

恢复测试

恢复测试

- ✓目标：验证系统从软件或者硬件失败中恢复的能力。
- ✓方法：在人为使发生系统灾难（系统崩溃、硬件损坏、病毒入侵等）的情况下，检查系统是否能够恢复被破坏的环境和数据。

用户界面测试

- **也称易用性测试，测试用户界面是否易于使用且具有较强实用性**
  - ✓是否把常用的按钮或菜单项放在醒目的位置
  - ✓软件的输出是否清晰、有序
  - ✓错误信息是否直观和易于理解
  - ✓软件错误顺序是否符合用户的业务过程
  - ✓界面的风格是否一致

兼容性测试

兼容性测试

- ✓目标：测试应用对其他应用或者系统的兼容性
- ✓方法：
  - 错误猜测

可用性测试

可用性测试

- ✓目标：检查系统界面和功能**是否容易学习**、使用方式**是否规范一致**，是否会误导用户或者使用模糊的信息，一般与功能测试结合使用
- ✓方法：可以采用用户操作、观察(录像)、**反馈并评估的方式**

验收测试

- ✓ 目标：确保系统达到预期的交付标准，验证新建系统产品是否能够满足用户需求
- ✓ 方法：一般需要结合回归测试，并谨慎处理新出现的Bug

α测试和β测试

❑α测试

- ✓**软件开发公司组织内部人员**模拟各类用户行为对即将面市的软件产品（称为α版本、内部测试版）进行测试，发现错误并修正
- ✓尽可能逼真地**模拟实际运行环境和用户**对软件产品的操作，并尽最大努力涵盖所有可能的用户操作方式
- ✓经α测试并进行修改后的软件产品称为**β版本**（也称外部测试版）

❑β测试

- ✓软件开发公司组织各方面的**典型用户**在日常工作中实际使用β版本，或为对外进行宣传而将β版本免费赠送给典型用户（很多情况下，β版本可以通过Internet免费下载，也可以向软件公司索取），并要求用户报告异常情况、提出批评意见
- ✓β测试是在与开发者无法控制的环境下进行的软件现场应用

回归测试

- **修改程序可能会引入新的错误**
  - ✓原先“正常”的程序现在变得“不正常”
- **回归测试目的**
  - ✓验证软件新版本是否从正常状态回归/退化到不正常状态
- **方法**
  - ✓重新进行测试，再次运行所有的测试用例来发现缺陷
- **回归测试最好能够自动化**
  - ✓单元测试是回归测试的基础



Part 4

软件测试管理

测试过程与测试文档

- ✓ 测试团队由测试经理、测试设计人员、测试执行人员三部分人组成。
- ✓ 测试经理负责测试组织与规划，搭建测试环境，监督测试过程。
- ✓ 测试设计人员负责将软件需求转化为测试需求，将测试需求转化为测试用例，并且在必要时设计与实现一些小的测试工具。
- ✓ 测试执行人员，按照测试设计文档，负责具体的测试用例的实施与执行，并记录测试结果，书写测试报告。
- ✓ 产生测试计划之后，接下来就是找个测试模板，完成填空而已。测试计划的编写时间，开始于需求分析阶段之初，结束于软件设计阶段之时。

成立软件测试组织

- 软件测试是一项独立性的工作
- 成立单独的软件测试小组
  - 成员由各个软件测试工作师组成
  - 高效的开展软件测试，确保软件测试工作的权威性
  - 软件测试小组也不受软件开发小组的管理，以确保软件测试的独立性和结果的客观性
- 需在软件开发早期就成立软件测试小组
  - 以便他们尽早地介入到软件测试工作之中，包括开展必要培训、了解软件项目的整体情况、掌握软件需求、制定软件测试计划等

软件测试的输出

- 软件测试计划
- 软件测试报告，记录软件测试情况以及发现的缺陷
  - 软件单元测试报告
  - 软件集成测试报告
  - 软件确认测试报告
  - 系统测试报告等
  - 每个报告详细

制定和实施软件测试计划（1/2）

- **标识符**：用以标识本测试计划
- **简介**：介绍测试的对象、目标、策略、过程和进程等方面的内容
- **测试项**：描述接受测试的软件元素，包括代码模块或质量属性
- **待测软件特征**：说明接受测试的软件特征，如软件需求等
- **免测软件特征**：说明无需接受测试的软件特征，如软件需求等
- **测试策略和手段**：说明对软件进行测试的策略和手段
- **测试项成败标准**：说明每个测试项通过软件测试的标准
- **测试暂停/停止的标准**：说明软件测试暂停或停止的决策依据

制定和实施软件测试计划（2/2）

- **测试交付物**：说明测试过程中或完成之后应该交付的软件测试制品
- **测试任务**：描述测试的主要任务
- **测试环境**：描述测试所需建立的环境
- **职责**：说明每项测试任务的主要负责人或团队及其职责
- **进度安排**：描述测试的过程及时间安排
- **成本预算**：估算软件测试的成本
- **风险与意外**：描述测试过程中可能存在的风险和可能发生的意外

软件测试八项流程

产生测试计划包括五个基本步骤：熟悉项目情况、确定和排序测试需求、定义测试策略、估计测试工作量和配置测试资源。

软件测试过程，应该包括如下八项流程：

1. 熟悉项目情况

测试经理必须充分了解了被测试项目的情况，如何体现充分？可以不断积累问题列表，衡量自己是否真正了解被测项目，例如：

(1). 产品的运行平台和应用领域；

(2). 产品使用者的特点；

(3). 产品主要的功能模块；

(4). 测试的目的和侧重点；

(5). 被测软件的数据是如何传递、存储的；

(6). 产品采用的实现技术；

(7). 同类型产品有哪些，各自的特点和不足；

(8). 该产品的发布，是否被公司看作是一项非常重要而关键的事情；

(9). 产品的前期设计和开发工作，是否有资深的技术人员主导或参与；

(10). 项目开发负责人是谁，主要开发人员都有谁，各负责哪些部分，如何联系；

(11). 项目计划中安排的测试时间是多少；

(12). 开发团队工作进展如何；

(13). 测试人员目前各自的工作量如何。

2. 排序测试需求

- 在测试设计人员的协助下，重点确定测试需求并赋予其优先级。
- 将需求对客户的重要性作为一个排序标准，客户最关心的、重点强调的、是必须测试的，即使时间来不及，也要想办法和客户协调争取测试时间，因为关键需求没有充分测试，会影响公司在客户心目中的形象，那将比推迟提交软件的代价更惨重

测试需求排序示例

- 下表给出了图书馆信息系统14个功能点的测试优先级排序，供参考。

编号	功能名称	功能描述	供参考的测试优先级
1	图书入库信息录入	给图书分类编号，并录入系统	1
2	读者信息录入	录入读者基础信息	2
3	图书借阅信息录入	录入读者借阅图书信息	3
4	图书归还信息录入	录入读者归还图书信息	4
5	图书罚款信息录入	录入读者罚款图书信息	11
6	图书注销信息录入	录入注销图书信息	12
12	查询读者信息	录入查询读者信息	5
8	查询图书信息	录入查询图书信息	6
9	读者网上注册	录入读者网上注册信息	8
10	读者网上登陆	录入读者网上登录信息	9
11	读者网上查询图书	录入读者网上查询图书信息	10
12	图书订购	录入订购图书信息	13
13	图书借还统计	统计图书资源的利用情况	12
14	补办借书卡	作废原借书卡并补办新借书卡	14

3．定义测试策略

- ✓ 测试需要考虑模块、功能、性能、接口、版本、配置和工具等各个因素的影响。
- ✓ 测试策略主要包括：测试目的、测试用例、测试方法、测试通过标准和特殊考虑。
- ✓ 一个测试功能点，要定义一个测试策略项
- ✓ 测试策略项中包括了详细的测试信息，测试执行人员参照它，就可以进行实际测试了。

3．定义测试策略

下表中，是图书管理系统验证登录界面、输入框设置是否合理的测试策略项的具体定义。

测试功能点编号	XXV1.0-010-018
测试目的	测试读者网上登录界面的输入框（用户名和密码），大小设置是否合理
测试阶段	系统测试
测试类型	功能测试
测试方法	手工测试
测试用例	输入允许的最长用户名和密码； 输入比允许的最长用户名和密码多一位的字符
通过标准	小于等于允许的用户名和密码长度时，输入框能够完全显示内容； 大于允许的用户名和密码长度时，输入框不给予显示
特殊考虑	无

4．设计测试用例

对应一个测试策略项，可能需要设计多个测试用例。

设计测试用例，可以采用如下的一些技术：

(1). 对输入数据进行等价类划分。

(2). 为每一个等价类，分别设计出通过测试用例和失败测试用例。

(3). 绘制状态转换图，分析状态测试，设计状态测试用例。例如，分析出可能的状态，以及从一种状态转入另一种状态所需的输入、条件、状态变量及输出结果。

(4). 从压力条件、重负条件两个方面考虑，设计测试用例，寻找存在的竞争条件。

(5). 坚持二八定律，测试工作绝对不能安排得前松后紧。

(6). 积累经验，凭借直觉，设计测试用例。

5．估计测试工作量

(1) 要确定一个测试项目的测试需求点的数量；

(2) 要验证一个测试需求点包括几个关键测试动作；

(3) 要确定执行一个关键测试动作所需的时间 （执行步骤可以用平均时间粗略计算）；

(4) 依次累计求和，就可以粗略估计出一个测试项目（或者估计出几个测试需求点的测试工作量）。

5．估计测试工作量

一个测试项目工作量的粗略估算方法为：

$$\sum_{j=1}^{n}\sum_{i=1}^{m}\text{测试动作 ij所需的时间}$$

- ✓ 其中：
- ✓ i代表一个测试需求点中的一个测试动作
- ✓ m代表一个测试需求点有m个测试动作
- ✓ j代表测试项目中的一个测试需求点
- ✓ n代表测试项目有n个测试需求点
- ✓ 此外，对软件项目工作量进行估计的Delphi法、类比法、功能点估计法、无礼估计法，在某种程度上也适用于软件测试工作量的估计。

6．配置测试资源

- ✓ 主要是将测试任务和需要的测试资源，分配给具体的测试执行人员。避免出现如下情况发生：
  - A说“我测试过了”，B说“我以为没有测试，我也测试了”；
  - A说“你没有测试吗？”，B说“我以为你测试过了”。
- ✓ 最终，测试经理将思考结果记录下来，填写测试计划模板，到此并不代表测试计划编写完成。
- ✓ **如何验证测试计划可行性**，主要是**测试任务分配**部分。因此，要将测试计划在测试团队内进行评审，当然有开发部门人员参与更好，他们能从开发角度分析测试计划的可行性。总之，**只有通过评审的测试计划，才能被执行**。

7．执行测试

- ✓ 上述六项程序完成后，软件测试人员对项目组提交的软件工作产品或软件最终产品，就**执行软件测试**。
- ✓ 执行的**基本方法**是按照设计好的测试用例，**一个用例接着一个用例地进行测试**，**记录**每个用例的**执行结果**，并将执行结果与预期结果比较，一旦发现偏差，就得进行分析，找出偏差原因，确定该偏差是否由软件缺陷所致。
- ✓ 若是缺陷所致，则应尽量确定该缺陷的名称、类型、位置、场景、发现时间、产生原因、危害后果。

8．书写测试报告

测试报告用来对测试结果进行分析说明，经过测试后，证实了软件具有的能力，以及它的缺陷和限制，并给出结论性意见，这些意见既是对软件质量的评价，又是决定该软件能否交付客户使用的依据。测试报告至少要包括几个方面的内容：

(1). 测试任务描述与测试环境说明

(2). 测试版本比较和测试方法

(3). 功能测试描述、性能测试描述和确认性测试描述

(4). 遗留问题描述

(5). 测试总结